

Redesigning Jaltantra: System migration and performance enhancement

A Masters Thesis Report Submitted in fulfillment of the
requirements for the degree of
Masters of Technology
by

Harikrishna Shenoy (22M0759)

under the guidance of

Prof. Om Damani

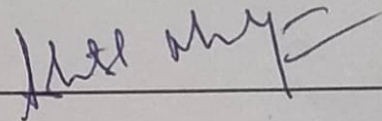


Department of Computer Science and Engineering
Indian Institute of Technology, Bombay
Mumbai 400 076

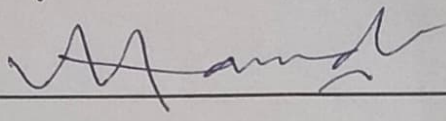
Dissertation Approval

This dissertation entitled **Redesigning Jaltantra: System migration and performance enhancement** by **Harikrishna Shenoy** is approved for the degree of Master of Technology.

Examiners: Prof. Ashutosh Mahajan:



Prof. Ranade.

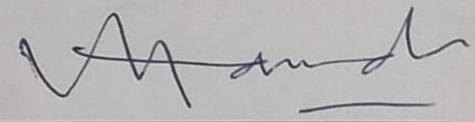


Supervisor: Prof. Om DAMANI

O. P. Damani

Prof. Ranade.

Chairperson:



Date: 14 June 2024

Place: IIT Bombay

DECLARATION

I declare that this written submission represents my ideas in my own words and where others ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date: 14 June 2024

Place: IITB

A handwritten signature in black ink, appearing to read 'H. Shenoy', with a stylized flourish at the end.

(Harikrishna Shenoy)

(22M0759)

Abstract

JalTantra is a web-based tool that employs both linear programming for branched sections and nonlinear programming for looped sections to optimize water distribution network costs. It achieves the optimum solution by solving the complex problem of selecting the most efficient pipe diameters throughout the network.

JalTantra has robust solver framework, however the user interface, server and deployment architecture is outdated and prone to vulnerabilities thus needing significant improvement.

This thesis aims to optimize the web application's architecture by migrating it to spring boot , simplifying development and deployment, fixing bugs and vulnerabilities associated with user interface and adding new features to the application. This work has the potential to greatly benefit application users by offering a more efficient and cost-effective web-application for water network design.

Contents

1 Introduction	1
1.1 Architecture	1
1.1.1 User interface	1
1.1.2 Middleware	2
1.1.3 Back-end	3
1.1.4 Deployment	4
1.2 Way-ahead	5
2 UI Vulnerabilities	6
2.1 Introduction	6
2.2 Solver launching bug	7
2.2.1 Root causes	7
2.2.2 Temporary solution	8
2.2.3 Long term solution	9
2.3 Validation Bugs	9
2.3.1 Disconnected nodes	9
2.3.2 Duplicate links	13
2.4 Caching bugs	15
2.4.1 Solver status	16
2.4.2 Launcher error	16
2.5 Way-ahead	17
3 Codebase challenges	18
3.1 Introduction	18
3.2 Dependency on JAVAX libraries and servlets	18
3.3 Low modularity in code base	19
3.3.1 SOLID principles	19
3.4 External Tomcat dependency	20

3.5	Manual configuration management	21
3.5.1	IDE support and dependency management	21
3.6	Additional challenges	22
3.7	Towards solution frameworks	22
3.7.1	Jakarta EE	23
3.7.2	Django	23
3.7.3	NodeJS	24
3.7.4	Spring boot	24
3.8	Way-ahead	25
4	Architecture redesign	26
4.1	Introduction	26
4.2	Dependence on outdated Javax servlets and libraries	26
4.3	Making code-base modular	31
4.3.1	Breaking down code into modules-Example	32
4.3.2	Wiring classes at runtime	33
4.3.3	Benefits of Spring boot's dependency injection	35
4.4	Configuration management	37
4.4.1	Challenge	37
4.4.2	Implementation	37
4.5	Removing external tomcat dependency	40
4.6	Granular logging	41
4.7	Build automation	43
4.7.1	Maven	43
4.8	Migration workflow	44
4.9	Way-ahead	46
5	Feature enhancement	47
5.1	User management and security	47
5.2	Implementing user management	49
5.2.1	Login and register Pages	49
5.2.2	Defining database schema	50
5.2.3	Persisting user details	51
5.2.4	Protecting URLs using spring security	55
5.2.5	Email verification	56
5.3	Deployment automation	58
5.3.1	Using Maven Wrappers and Bash Scripting	58

5.3.2	JSON configurable parameters	59
5.4	Automated monitoring	59
5.4.1	Motivation	60
5.4.2	Solution	61
6	Conclusion and Future Work	62
6.1	Conclusion	62
6.2	Future Work	63

List of Figures

1.1 Jaltantra UI	2
1.2 Front end	3
1.3 Middleware and Backend	3
2.1 Bug Categories	7
2.2 Linux Users	8
2.3 Linux Users Table	8
2.4 Incorrect Notification	10
2.5 Disconnected graph with loop passed validation	10
2.6 Buggy code patch	11
2.8 Corrected notification	12
2.7 Code Patch to resolve bug	12
2.9 Incorrect Notification	13
2.10 Correct notification for duplicate links	14
2.11 Code patch for duplicate link detection	15
2.12 Various solver status	16
2.13 Cleaning up launcher error	17
4.1 Javax libraries removed	27
4.2 Spring MVC	27
4.3 Inflexible coding style in web-servlet, all if conditions need to be handled in same method	30
4.4 Extra boiler plate required in web.xml to configure the servlet	30
4.5 Flexible decoupled coding in MVC	30
4.6 Non modular class	33
4.7 Modular class design	33
4.8 Dependency Injection	34
4.9 Dependency without annotation	36
4.10 Dependency with annotation	36

4.11 Application properties file- Development environment	38
4.12 Application properties file- Deploy environment	38
4.13 @Value annotation	39
4.14 Values injected using Thymeleaf template engine	39
4.15 External Tomcat dependency	40
4.16 Logging properties	42
4.17 Console Interceptor	42
4.18 Manual build process	43
4.19 Maven working	44
5.1 Securing end points	48
5.2 Jaltantra Login	49
5.3 Jaltantra Registration	50
5.4 Database Schema	51
5.5 Trigger example	54
5.6 User session data	54
5.7 Verification Email	55
5.8 Security Classes	56
5.9 Verification Email	58
5.10 Verification successful	58
5.11 Application monitoring	60

Chapter 1

Introduction

Water distribution systems are essential to modern infrastructure that delivers clean and safe drinking water to communities worldwide. These systems consist of a net-work of pipes, valves, pumps, and storage tanks that transport water from treatment plants to homes, businesses, and other facilities. The main objective of the Water Distribution System is to provide the necessary amount of water to demand nodes satisfying cost and pressure constraints. Jaltantra 2.2.2.0 is a web-app based solution , which provides a user interface to take inputs about various parameters about network problem to solve, then it uses linear and non-linear programming [4] using solvers like Knitro,Baron using python to manager the parallel sessions for above mentioned solvers.Jaltantra Linear and Non-linear programming based solvers are robust.

1.1 Architecture

This section will provide a high level overview of various components in Jaltantra Loop.UI is common for branch as well as loop version of it. web application.In Loop version the back-end solvers are python based whereas in branch version the solver is java based wrapper of google OR tools, OR-Tools suite provides operations research software libraries and APIs for constraint optimization, linear optimization, and flow and graph algorithm.

1.1.1 User interface

Functionalities of UI

- Jaltantra UI, is Javascript based and provides various options for users to load input as the network files in various formats like XML,Excel Branch file.
- Provides interactive ways to create network by using Google Map API.
- Options to save the results in Excel,XML format.
- Provides option to provide the amount of time the solve should run, currently the options are 1minute,5 minute and 1 hour
- UI displays the notification about status of the input parsing like validation error in input data,status of solvers like running,time left for .

Figure 1.1: Jaltantra UI

1.1.2 Middleware

The UI is served to end user on end points exposed by Java servlet. The servlet based middle ware is responsible for below functionalities:

- Parsing the input data , binding the data to objects like pipe, nodes etc which are part of Jaltantra optimizer functionality.
- Validating the data for the nodes connectivity,demands and other validation checks and sending the appropriate notification to UI for correcting it.

- After validation is successful, the middle-ware creates the data file , creates a hashed identifier for data which is used for caching and sends it to the back-end solver.
- It then retrieves the results and then sends it back to the UI.

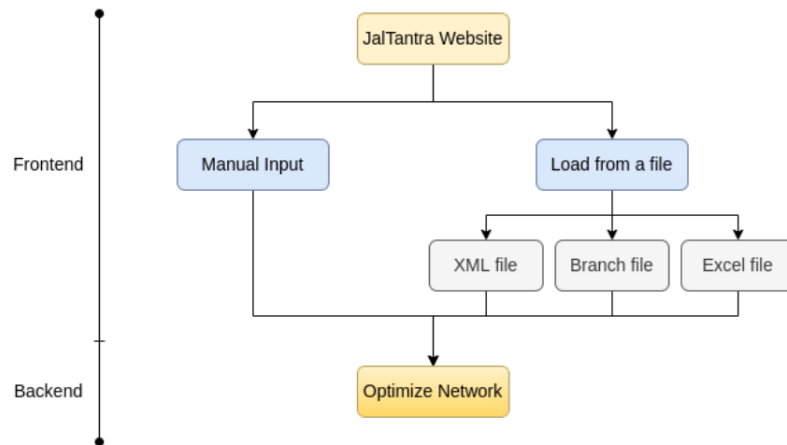


Figure 1.2: Front end

1.1.3 Back-end

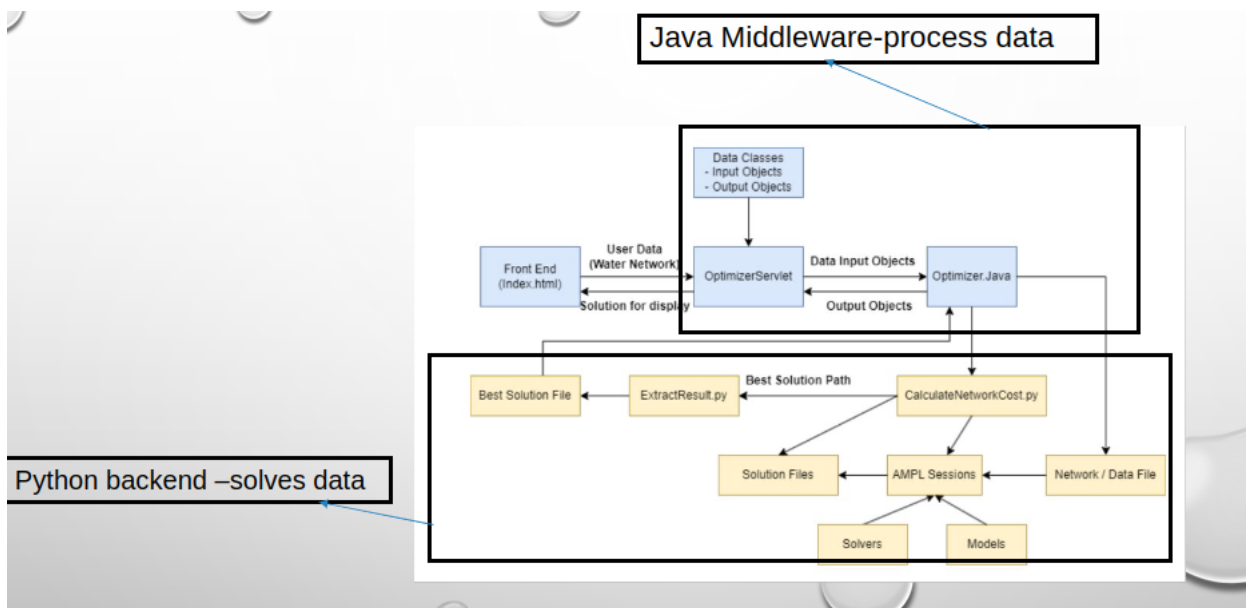


Figure 1.3: Middleware and Backend

- Back-end consists of python wrappers around the AMPL based Knitro and Baron solvers, the data file from middle-ware is stored in file-system.
- After this python script is invoked,invoking python script required TMUX session support.
- The Jaltantra web app which runs inside tomcat servlet container invokes TMUX sessions.
- TMUX is a terminal multiplexer that allows user to create and manage multiple terminal sessions within a single window or terminal emulator.
- It's particularly useful for managing multiple terminal sessions simultaneously this feature is used in invoking parallel sessions for solvers .
- After solvers have completed running i.e either the user defined time-limit is over or solver has finished solving the network (feasible or infeasible), the python scripts store the output in file system in the folders identified by the hash of input.

1.1.4 Deployment

Compiling code-base and building it into artifact and deploying it is part of software life-cycle.

- Jaltantra code classes and static files(html,css,javascripts etc) are compiled and packaged together in a WAR file.
- The compilation can be done only on specific IDE (IntelliJ 2021) which is a limitation , and will be described in detail in next chapters.
- After WAR file is built it is deployed via external tomcat server.
- Tomcat server configurations are done manually the configurations include ports and application context.

This process is manual and tightly coupled with the IDE version, reducing the flexibility of the development process.

1.2 Way-ahead

In the next chapters bugs and fixing of bugs related to application user interface will be described in detail, followed by describing challenges related to code-base and then migrating it to Spring boot, and explaining the features adopted from spring boot for efficient migration.

Chapter 2

UI Vulnerabilities

2.1 Introduction

Jaltantra Loop and Branch version have robust solver programs and being used by engineers since 2014 (branch version). However user interface had some vulnerabilities and bugs. All the bugs and vulnerabilities described in this chapter are part of Jaltantra Loop version. Bugs, defined as errors or flaws within a program, result in incorrect or unexpected outputs. These anomalies significantly hinder the user experience, making their resolution a critical priority in software development. Ensuring the smooth functioning of software is not merely a technical necessity but a pivotal component of maintaining user satisfaction and trust. Unresolved bugs can lead to frustration among both users and developers, reduced productivity, and potentially severe operational disruptions, underscoring the imperative need for efficient and proactive bug resolution strategies in the software development life-cycle. Below chart describes broad categories of bugs resolved, all these will be explained in detail in next sections, for each of the bugs first the main issue /fault will be highlighted then deep diving into root cause and concluding by highlighting the solution to resolve the bugs.

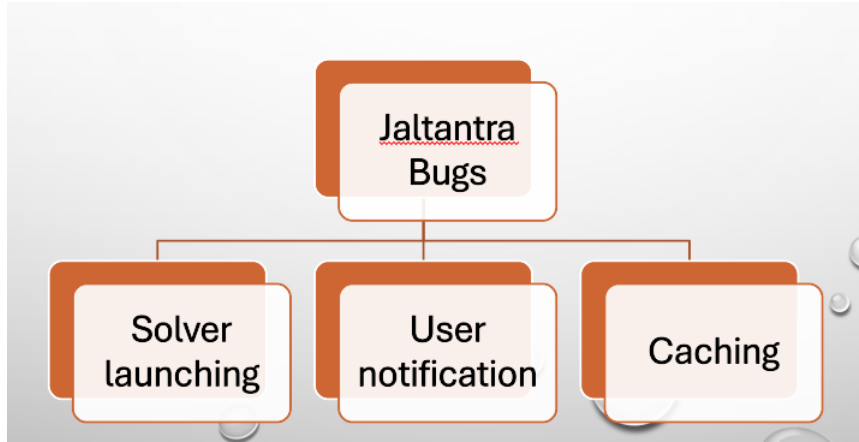


Figure 2.1: Bug Categories

2.2 Solver launching bug

As mentioned in introduction section, the jaltantra loop application was deployed as WAR file on tomcat program running on the deployment server as described in previous chapters. Java web-servlets deployed on tomcat handled requests on endpoints (on /optimize), however as servlets called solvers i.e back-end python wrappers to start solver session and pass the data to the Non-linear programming solver, the requests were actually being made by Tomcat container, this led to below is the summary of Issues.

- It was observed several times that the application was not able to launch the python based solver programs which relies on TMUX sessions being invoked when running on deployment server.
- However, there was no issue in invoking TMUX session, when application was tested in development environment, which was using tomcat from IntelliJ IDE.

2.2.1 Root causes

- As mentioned due to the WAR files were deployed on tomcat program in deployment environment, the 'user' invoking the Jaltantra code was 'tomcat' user.
- Further deepdive was done about various users using /etc/passwd file, which

is a system file commonly found on Unix and Unix-like operating systems such as Linux. It stores essential information about user accounts on the system.

- Each line in the file represents a single user account and contains several fields separated by colons (:). The fields typically include: Username, Password (or a placeholder indicating that the password is stored in a separate shadow file), User ID (UID), Group ID (GID), User Info (typically the full name of the user), Home directory, Login shell.
- Login shell of tomcat user was '/bin/false' which means it cant open a interactive shell like bash, as only user with Login shell with '/bin/bash' can invoke interactive shells.

```
hkshenoy:x:1000:1000:hkshenoy,,,:/home/hkshenoy:/bin/bash
tomcat:x:1001:1001::/opt/tomcat:/bin/false
mysql:x:128:136:MySQL Server,,,:/nonexistent:/bin/false
fwupd-refresh:x:129:137:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
```

Figure 2.2: Linux Users

User	hkshenoy	tomcat
Password	x	x
User ID	1000	1001
Group ID	1001	1001
User info	hkshenoy,	
Home Directory	/home/hkshenoy	/opt/tomcat
Login Shell	/bin/bash	/bin/false

Figure 2.3: Linux Users Table

2.2.2 Temporary solution

Manually the Login shell of tomcat user was changed to '/bin/bash' and this solved the deployment issues. However setting the login shell for the Tomcat user to /bin/bash is not recommended from a security standpoint. Tomcat is typically run as a service account with limited privileges, and granting it a login shell introduces unnecessary security risks as follows:

- Exposure to System Resources: Granting a login shell to the Tomcat user exposes it to potential exploitation by attackers. If an attacker gains unauthorized access to the Tomcat account, they could potentially escalate their privileges and access sensitive system resources.

- **Increased Attack Surface:** Allowing Tomcat to have a login shell expands the attack surface of the system. Attackers can exploit vulnerabilities in the shell environment to execute arbitrary commands, manipulate files, or perform other malicious activities.
- **Violation of Principle of Least Privilege:** Granting unnecessary privileges to the Tomcat user violates the principle of least privilege, which states that users should only have the minimum level of access required to perform their tasks. Giving Tomcat a login shell goes against this principle and increases the risk of unauthorized access and misuse.

2.2.3 Long term solution

Above patch of giving higher privilege to tomcat was a temporary fix as application did not have login/user details anyway at this point, and after migrating the spring boot application will be deployed as standalone application with embedded server and user of this running application in Linux based server will be the root user hence will by default have the shell privileges, thus permanently fixing the bug in deployment. More about this will be discussed in 'Migration' chapter

2.3 Validation Bugs

This section describes validation bugs. Validation is an important step in the data processing of Jaltantra middle ware, which intercepts the data coming from UI and creates the objects i.e pipes, nodes, arcs etc to act on the data, after mapping the objects from raw data, data is validated and then sent to the back-end python solvers for getting the solution.

2.3.1 Disconnected nodes

Accurate validation of user inputs is critical in network analysis/solving applications. The issue of user inputs containing disconnected nodes passing validation checks poses significant challenges. These inputs, when sent to solvers, result in failed attempts to solve disconnected networks. This not only leads to erroneous notifications about data in-feasibility but also impacts user trust and system reliability. The necessity to enhance validation mechanisms to accurately identify and

notify users about disconnected nodes is important for maintaining the integrity and usability of the application

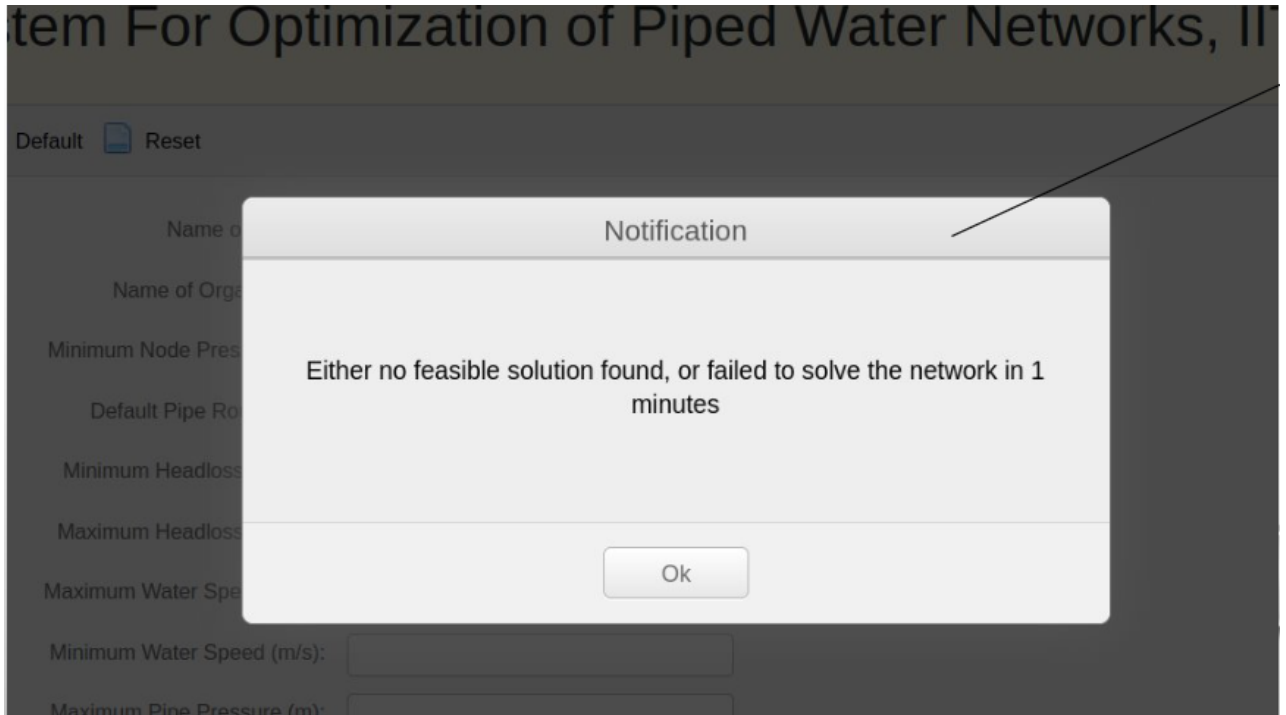


Figure 2.4: Incorrect Notification

Below are steps involved in resolving the bugs

- Issue Identification:
 - Symptom: Users received incorrect notifications indicating that the input data was not feasible.
 - Root Cause: The validation checker incorrectly passed inputs containing disconnected nodes because the presence of a loop was mistakenly considered sufficient for validation success. Incorrect piece of code below

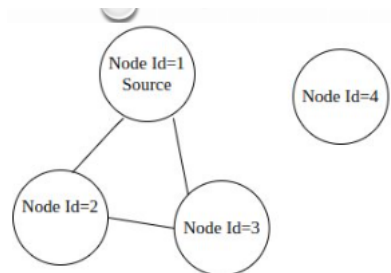


Figure 2.5: Disconnected graph with loop passed validation

```

HashSet<Node> seen = new HashSet<>();
Stack<Node> left = new Stack<>();
left.add(root);

while (!left.isEmpty()) {
    Node top = left.pop();
    if (seen.contains(top)) {
        return 2; // cycle
    }
    seen.add(top);
    for (Pipe pipe : top.getOutgoingPipes()) {
        left.push(pipe.getEndNode());
    }
}

```

Figure 2.6: Buggy code patch

- Investigation:
 - Back-end Log Analysis: Detailed examination of back-end logs revealed that the infeasibility was due to incorrect data—specifically, nodes with no incoming or outgoing connections (pipes).
 - Data File Examination: The data files contained nodes that were isolated, lacking connections to the rest of the network but had demands to be satisfied, which led to solver failures.
- Solution Implementation:
 - Validation Checker Enhancement: The validation checker was modified to include a check for disconnected graphs. This involved ensuring that all nodes are visited during the validation process.

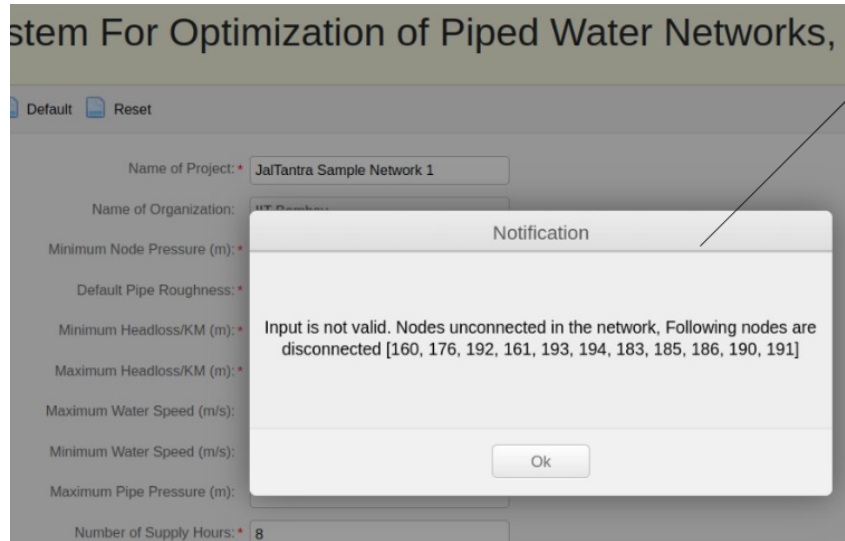


Figure 2.8: Corrected notification

```
//Function to check and populate the disconnected nodes in the network
private void PopulateDisconnectedNode(HashSet<Node> seen){

    disconnectedNodes=new HashSet<Integer>();
    for (Map.Entry<Integer, Node> mapElement : nodes.entrySet()) {
        Integer key = mapElement.getKey();
        Node node = mapElement.getValue();
        if(seen.contains(node) == false){
            disconnectedNodes.add(node.getNodeID());
        }
    }
}
```

Figure 2.7: Code Patch to resolve bug

- User Notification: The system was updated to notify users explicitly about disconnected nodes, providing clear and actionable feedback.
- Technical Approach: Graph Traversal Check: Implemented a recursive traversal algorithm Depth-First Search on undirected graph to ensure all nodes in the input data are connected.
- Validation Logic Update: The validation logic now returns success only if every node is visited during traversal. If any node is found to be disconnected, the validation fails, and users are informed about the specific nodes causing the issue.

- Users receive precise notifications about the nature of the validation failure, specifically identifying disconnected nodes, which aids in quick rectification of input data. Increasing reliability of system.

2.3.2 Duplicate links

The issue of user inputs containing duplicate links passing validation checks is a significant problem. When such data is processed by solvers, it results in infeasible solutions, causing incorrect notifications to users about the infeasibility of their problem. To maintain system reliability and user trust, it is essential to detect and handle duplicate links during the validation phase, providing precise feedback to users.

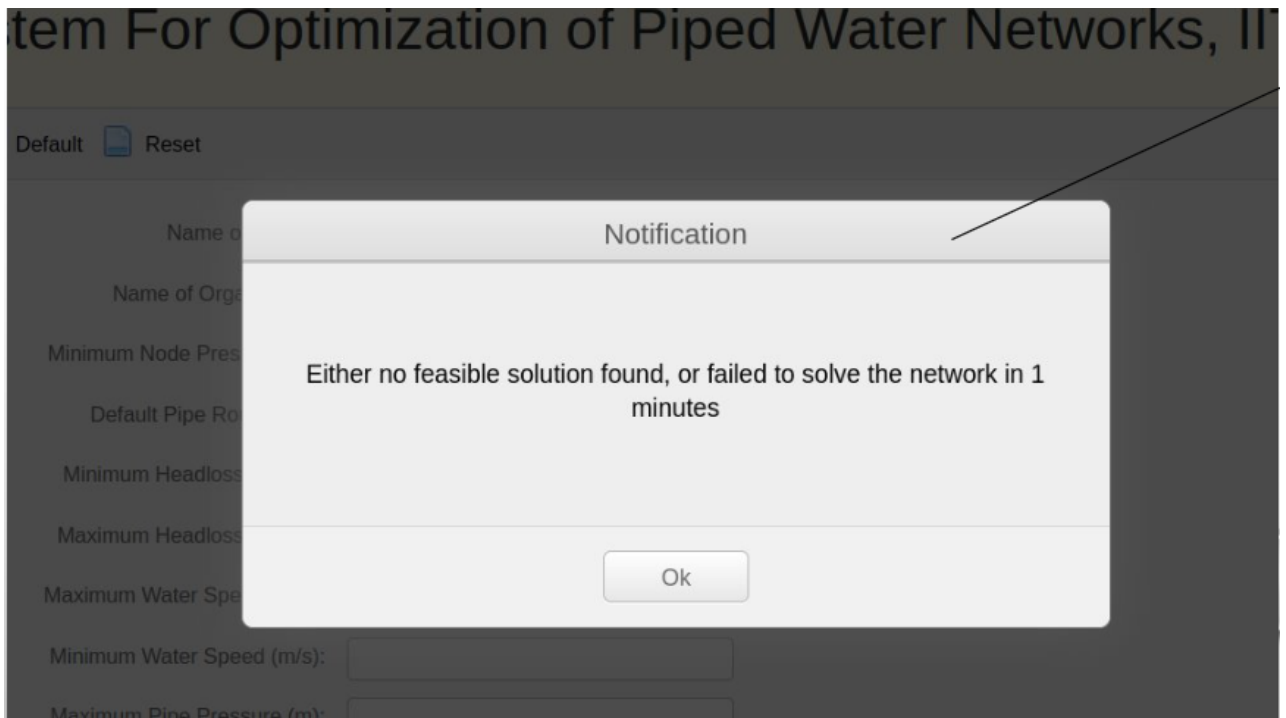


Figure 2.9: Incorrect Notification

- Issue Identification:
 - Symptom: Users received incorrect notifications stating that their problem was not feasible.
 - Root Cause: The validation checker failed to detect duplicate links in the input data, leading to erroneous solver outputs.

- Investigation:
 - Back-end Log Analysis: Examination of back-end logs revealed that the infeasibility of solutions was due to data containing duplicate links.
 - Manual Data Inspection: A detailed manual inspection of the data files confirmed the presence of duplicate entries, which the validation checker overlooked.

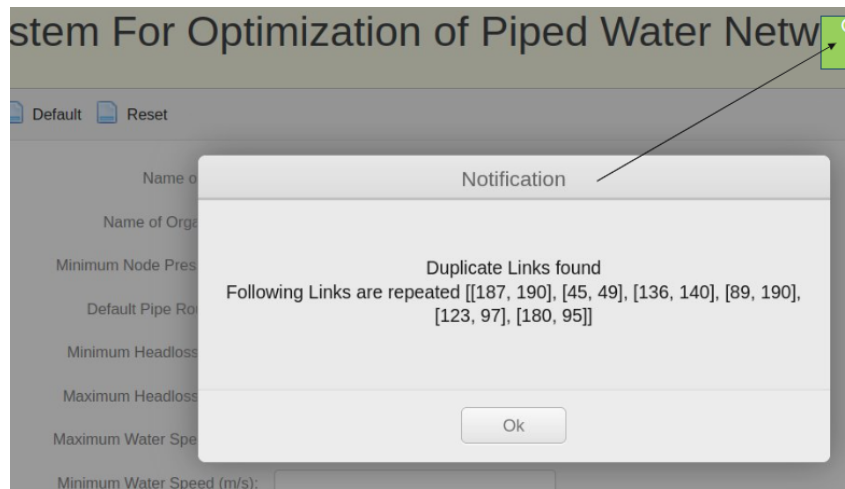


Figure 2.10: Correct notification for duplicate links

- Solution Implementation:
 - Duplicate Link Checker: A new validation step was introduced to check for duplicate links in the input data. This involved using a HashSet to identify and flag duplicates during the parsing process.

```

private boolean checkDuplicateLinks(){
    HashSet<List<Integer>> linkSet=new HashSet<>();
    duplicatedLinks=new HashSet<List<Integer>>();

    for (Map.Entry<Integer, Pipe> mapElement:pipes.entrySet()){
        Pipe pipe=mapElement.getValue();
        int startNode=pipe.getStartNode().getNodeID();
        int endNode=pipe.getEndNode().getNodeID();

        if(linkSet.contains(List.of(startNode,endNode))==true){
            duplicatedLinks.add(List.of(startNode,endNode));
        }

        linkSet.add(List.of(startNode,endNode));
    }

    if(duplicatedLinks.size()>0){
        return true;
    }
    return false;
}

```

Figure 2.11: Code patch for duplicate link detection

- User Notification: The system was updated to notify users specifically about duplicate links, providing clear and actionable feedback.
- Validation Logic Update: The validation logic was enhanced to return success only if no duplicates are found. If duplicates are detected, the validation fails, and users are informed about the specific duplicate links.

2.4 Caching bugs

To enhance performance and efficiency, Jalantra has caching mechanisms implemented. Cache directories are stored in the file-system and are identified by a hash of the input file contents. This ensures that identical input data does not trigger redundant solver runs. Instead, the system can quickly retrieve results from the cache, significantly reducing computation time and resource usage. However, a cache miss occurs when the input data does not match any cached entries, prompting the solver to run anew and generate fresh results.

2.4.1 Solver status

The solver status provides crucial feedback on the progress and outcome of the solver's operations, solver statuses include:

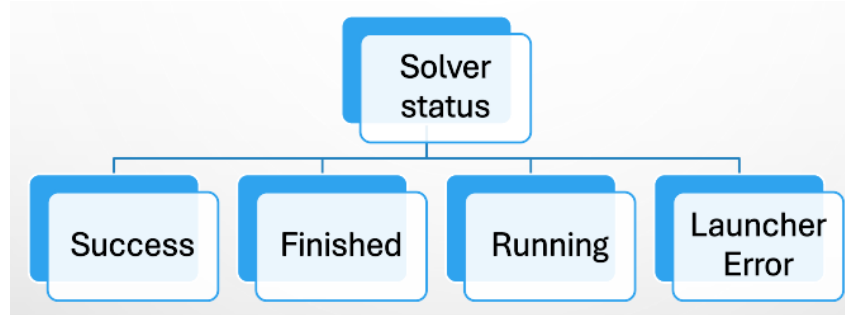


Figure 2.12: Various solver status

- **Success:** Indicates that the solver has successfully solved the problem with the given data.
- **Finished (No Feasible Solution):** The solver has completed processing but found no feasible solution.
- **Running:** The solver is still processing and requires more time to find a solution.
- **Launcher Error:** The solver failed to launch due to issues such as licensing problems, network connectivity issues, or other external factors not related to the input data.

2.4.2 Launcher error

The presence of launcher errors, which prevent the solver from starting due to transient issues such as licensing or network connectivity, poses a significant challenge to caching. These errors, unrelated to input data, result in incorrect caching, which in turn prevents users from re-triggering the solver once the issue is resolved. Enhancing the validation mechanism to properly address launcher errors and manage the cache is essential for maintaining system reliability and user trust. So it uses up space in server file-system and also prohibits users from restarting the server

To address the issue of launcher errors, a comprehensive solution was implemented, involving the following steps:

- **Error Analysis:** Launcher errors were identified as issues related to external factors such as licensing or network connectivity, which are not related to the input data.
- **Caching Mechanism:** Analysis revealed that the current caching mechanism retained files associated with launcher errors, leading to cache hits on subsequent attempts and preventing new solver runs.
- **Solution Implementation: Result and Status File Deletion:** A new approach was introduced to delete result files, status files, and folders associated with launcher errors. This ensures that no invalid data is cached.

```
final String status = readFileAsString( pathToFile: pathToNetworkSpecificDirectory + "/0_status").strip();
if (status.startsWith("Launch_error")) {
    String directoryPath = pathToNetworkSpecificDirectory;

    // Create a File object representing the directory
    File directory = new File(directoryPath);

    // Check if the directory exists
    if (directory.exists()) {
        // Use the deleteDirectory method to recursively delete the directory
        boolean success = deleteDirectory(directory);

        if (success) {
            CustomLogger.log( str: "Network Specific Directory deleted successfully.");
        } else {
            System.out.println("Failed to delete network specific directory.");
        }
    } else {
        System.out.println("Network Specific Directory does not exist.");
    }
}
```

Figure 2.13: Cleaning up launcher error

- **Cache Miss Handling:** By removing these files, subsequent solver attempts will result in cache misses, triggering the solver again and allowing users to proceed once the transient issue is resolved.

2.5 Way-ahead

The bug fixing process pointed to various challenges in code-base which made development and fixing bugs cumbersome, next chapter describes the challenges in code-base in detail.

Chapter 3

Codebase challenges

3.1 Introduction

Jaltantra version 2.2.2.0 web app is deployed currently on IIT-B servers. High level overview of front-end and back-end have been described in previous chapter. During bug fixing various challenges and shortcomings were observed in Jaltantra code-base, this chapter describes these challenges and shortcomings in code-base which serves as motivation to migrate it to newer frameworks.

3.2 Dependency on JAVAX libraries and servlets

Jaltantra uses Javax.servlet and Javax.xml libraries, Javax libraries are outdated and has below issues, hence these need to be migrated to Jakarta or java name-space libraries.

- Lack of updates: Historically, Javax libraries have been slow to receive updates compared to other frameworks. This can lead to compatibility issues with newer technologies and security vulnerabilities.
- Vendor Lock-In: Dependency on Javax libraries can lead to vendor lock-in, particularly with application servers like Tomcat etc. This limits the portability of applications across different platforms and server versions.
- Heavyweight: Some Javax APIs are considered heavyweight, requiring lot of boiler plates. This can lead to bloated code-bases and decreased performance, due to increasing verbosity in code-base.

- Complexity: As mentioned in previous parts the APIs provided by javax libraries can sometimes be overly complex, especially for developers new to enterprise Java development. This complexity can increase the learning curve and make the development process and maintenance more cumbersome.
- Java EE to Jakarta EE transition: The transition from Java EE to Jakarta EE has caused some confusion and fragmentation within the Java ecosystem. While efforts have been made to maintain compatibility, developers may encounter issues when migrating existing applications to newer versions.

3.3 Low modularity in code base

Jaltnatra v2.2.2.0 code base is monolithic and classes are not following SOLID principles, this makes code difficult to maintain ,extend and migrate to newer frameworks, hence modularity of code-base was bottleneck in way of improving the system.Following is a brief explanation of SOLID principles.

3.3.1 SOLID principles

The SOLID principles are a set of five design principles that help developers create more maintainable, scalable, and robust software applications. Here's a brief overview of each principle:

- Single Responsibility Principle (SRP): A class should have only one reason to change, meaning it should have only one responsibility or job. This principle promotes high cohesion by ensuring that each class is focused on doing one thing well, making the code easier to understand, maintain, and test.
- Open/Closed Principle (OCP): Software entities (classes, modules, functions) should be open for extension but closed for modification. This means that you should be able to add new functionality to a system without modifying existing code. You achieve this by using abstractions, interfaces, and inheritance to allow for extension through sub-classing or composition.
- Liskov Substitution Principle (LSP): Objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program. In other words, derived classes should be able to substitute their base classes without altering the behavior expected by clients. This

principle ensures that inheritance hierarchies are well-designed and do not introduce unexpected behavior.

- Interface Segregation Principle (ISP): Clients should not be forced to depend on interfaces they do not use. Instead of implementing large, monolithic interfaces, you should design smaller, more specific interfaces tailored to the needs of clients. This prevents the proliferation of unnecessary dependencies and reduces coupling between components.
- Dependency Inversion Principle (DIP): High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions. This principle promotes loose coupling between components by decoupling them from concrete implementations and relying on abstractions or interfaces instead.

Current Jaltantra code-base does not follow Single responsibility principle making the code-base very difficult to extend and debug the solutions and implementations to overcome this issue will be described in migration chapter.

3.4 External Tomcat dependency

As described in previous chapter the WAR files are uploaded on Tomcat server on deployment machine. Tomcat needs to run as a process and has a user account in deployment machine, and manages all the back-end functionalities of the web app. Jaltantra version 2.2.2.0 is deployed on Tomcat 9. However as Tomcat updates it will also not support javax libraries, hence this introduces a hard dependency on tomcat version as well , for e.g Tomcat 10 will not support Jaltantra version 2.2.2.0 . This is not ideal software development practice, in situation where we will need to re-deploy app on some other physical server, we might need to install older tomcat version to make it compatible with Jaltantra, this makes whole server architecture prone to being outdated and not keeping up with latest changes. Hence in migration chapter , this dependency will be resolved and make Jaltantra more agile and compatible with latest library/package versions. Solution to above dependency will be described in migration chapter.

3.5 Manual configuration management

Configuring the software for the target environment by setting appropriate parameters, such as database connection strings, API endpoints, and environment-specific settings is essential step in application development and testing, below are some limitations in current structure of code with respect to configuration management.

- Currently Jaltantra uses Tomcat 9 as servlet container and tomcat requires to be configured manually in an environment before deploying, this makes the whole process of configuring deployment cumbersome.
- Changes to parameters like version, folder paths, require changes to code-base at many locations rather than being controlled by external configuration files which is preferred in software development process.
- Setting up initial python environment for developer to be able to run Jaltantra is a manual process, which is prone to error.

3.5.1 IDE support and dependency management

Currently Jaltantra Loop code requires IntelliJ version 21.1 (By JetBrains) and does not work on other versions/IDE hindering the flexibility of developers working on the project, also the code uses javax packages which are no longer supported by Java and are outdated, [6].

- Prepare WAR file: First, to packaging web application into a WAR file. This typically involves compiling Java code, organizing static resources (HTML, CSS, JavaScript, etc.), configuring deployment descriptor (web.xml), and any other necessary locations or scripts for application, all the above steps are done currently on IntelliJ 2021 version, and due to javax library dependency newer versions of IntelliJ are incompatible with it.
- Access the Tomcat Manager: Apache Tomcat typically comes with a web-based management interface known as the Tomcat Manager. This is run as a daemon process on the server, we need to access this interface through a web browser by navigating to `http://localhost:8080/manager/html` (assuming Tomcat is running on localhost and using the default port 8080)
- Login to the Tomcat Manager [1]: Manually put the appropriate credentials to access the Tomcat Manager. These credentials are typically configured in the `tomcat-users.xml` file located in the `conf` directory of your Tomcat installation.

- **Deploy the WAR file:** Once logged in, interface to deploy WAR files is visible. There should be an option to upload and deploy a new WAR file. this option is used to upload the WAR file prepared earlier.
- **Verify Deployment:** After the WAR file is uploaded, Tomcat will automatically deploy the web application. See the application listed in the Tomcat Manager interface. It will display the application name, context path, status, and other relevant information.
- **Access your application:** Once deployed, we can access web application by navigating to `http://localhost:8080/yourAppName`, where `yourAppName` is the context path specified in deployment descriptor or the name of your WAR file (minus the `.war` extension).

3.6 Additional challenges

- **Logging:** Tomcat maintains extensive logs for each deployed application. These logs can be useful for troubleshooting issues or monitoring the behavior of your application, can be found in `/opt/tomcat/logs` folder, however, better control on login like control on what events like warning, trace or general information to log, need to be controlled to control the log sizes, log rollover policy etc.
- **Security:** It's essential to secure Tomcat installation to prevent unauthorized access and ensure the integrity of your applications and data. This involves configuring user roles, access permissions as tomcat might expose another entry point for malicious users, hence it is added burden to ensure secure configuration for Tomcat.

3.7 Towards solution frameworks

To make the code base better with optimum developmental effort , support of a web framework is essential. A brief survey of various frameworks was done and suitable framework for migration was finalized.

3.7.1 Jakarta EE

Jakarta EE [5], formerly known as Java EE (Enterprise Edition), is a set of specifications that extend Java SE (Standard Edition) with specifications for enterprise features such as distributed computing and web services. Jakarta EE provides a structured environment for building large-scale, multi-tiered, scalable, and secure network applications, below are some drawbacks of Jakarta EE.

- Complexity: Jakarta EE is known for its steep learning curve and complexity. It requires a deep understanding of its extensive APIs and specifications.
- Configuration Overhead: Jakarta EE applications required a lot of XML configuration, which could become cumbersome and error-prone.
- Heavyweight: Jakarta EE applications are often considered heavyweight due to the extensive set of libraries and dependencies.
- Deployment: The deployment process can be complex, often requiring an external application server like Tomcat.
- Development Speed: The development process in Jakarta EE can be slower compared to more modern frameworks that provide more out-of-the-box functionality and conventions.

3.7.2 Django

Django [3] is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the "batteries-included" philosophy, providing numerous built-in features and tools, below are some drawbacks of Django.

- Monolithic: Django encourages a monolithic architecture, which can be a disadvantage when trying to build microservices or more granular, decoupled applications.
- Opinionated: Django is quite opinionated about how things should be done, which can limit flexibility and force developers to conform to its conventions.
- Performance: Django's performance might not match that of frameworks designed for highly concurrent and real-time applications.
- Template Engine: The Django template engine, while powerful, can be less intuitive and more restrictive compared to other template engines like jsp.

- **Security Concerns:** Despite its built-in security features, Django applications can still be vulnerable to common web vulnerabilities if not configured correctly. For example, issues can arise with Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) if not properly handled.

3.7.3 NodeJS

Node.js [7] is a runtime environment that allows to execute JavaScript on the server side. It uses an event-driven, non-blocking I/O model.

- **Callback Hell:** Node.js relies heavily on callbacks, which can lead to complex and hard-to-read code .
- **Single-Threaded:** Node.js is single-threaded, which can be a limitation for CPU-intensive tasks. While it handles I/O-bound tasks efficiently, CPU-bound tasks might degrade performance.
- **Ecosystem Maturity:** The Node.js ecosystem is vast and fast-growing, but it can sometimes lead to issues with unmaintained packages or security vulnerabilities in third-party modules.
- **JavaScript Limitations:** While JavaScript is versatile, it is not as mature for back-end development as some other languages, lacking certain features found in more traditional server-side languages.
- **Rapid Changes:** The Node.js ecosystem changes rapidly, which can lead to frequent updates and potential instability in dependencies.
- **Security Concerns:** Node.js applications can be vulnerable to security issues such as SQL injection, Cross-Site Scripting (XSS), and NoSQL injection if not properly handled. Additionally, the vast number of third-party packages can sometimes include insecure or outdated dependencies.

3.7.4 Spring boot

Spring Boot [9] is an extension of the Spring framework that simplifies the creation of stand-alone, production-grade Spring-based applications. It minimizes configuration and setup, allowing developers to get started quickly and focus on business logic. Below are some reason for choosing spring boot for migration.

- **Ease of Use:** Spring Boot simplifies the setup and configuration of Spring applications, providing defaults and reducing boilerplate code.
- **Familiarity of Use:** Already familiar with Java based frameworks and experience with spring boot.
- **Embedded Servers:** Spring Boot includes embedded servers like Tomcat, Jetty, or Undertow, making it easy to run applications directly without the need for an external server.
- **Micro-services Friendly:** Designed with micro-services in mind, Spring Boot is ideal for building micro-service architectures, allowing for decoupled architecture of development.
- **Community Support:** Spring Boot has a large and active community, ensuring continuous improvements and extensive support.
- **Integration:** Excellent integration with other Spring ecosystem projects and third-party libraries.
- **Security:** Spring Boot provides robust security features through Spring Security, allowing for easy implementation of authentication and authorization mechanisms.

3.8 Way-ahead

Due to challenges discussed and benefits spring boot provides, Jaltantra code base will be migrated to Spring boot. In next chapter, various benefits of spring boot framework will be illustrated with examples from migration process of the application.

Chapter 4

Architecture redesign

4.1 Introduction

Due to shortcomings of code-base as described in previous chapter, the web application will be migrated to Spring boot due to advantages provided by the spring framework. Jaltantra version 2.2.2.0 will be migrated to version 2.3.0.0. Next sections will describe various steps in migration and features of spring boot used to achieve the same. Overall, migrating the Java Servlet based codebase to Spring Boot 3.2[9] offers significant advantages in terms of development efficiency, maintainability, scalability, and security, ultimately resulting in a more robust and future-proof application for the client and in rapid speed, i.e the time required to successfully migrate the application will be less. Key goal of migration is to keep the application from user functionalities point of view same and improving the underlying architecture.

4.2 Dependence on outdated Javax servlets and libraries

Jaltantra web app used web-servlets and they utilized Javax name-spaces, these are outdated and are not compatible with latest IDE(Intelij ;2021) and build tools like Maven as Javax name-space has been deprecated from maven central repository, this makes code-base outdated and also difficult to extend with the latest features in web technology.

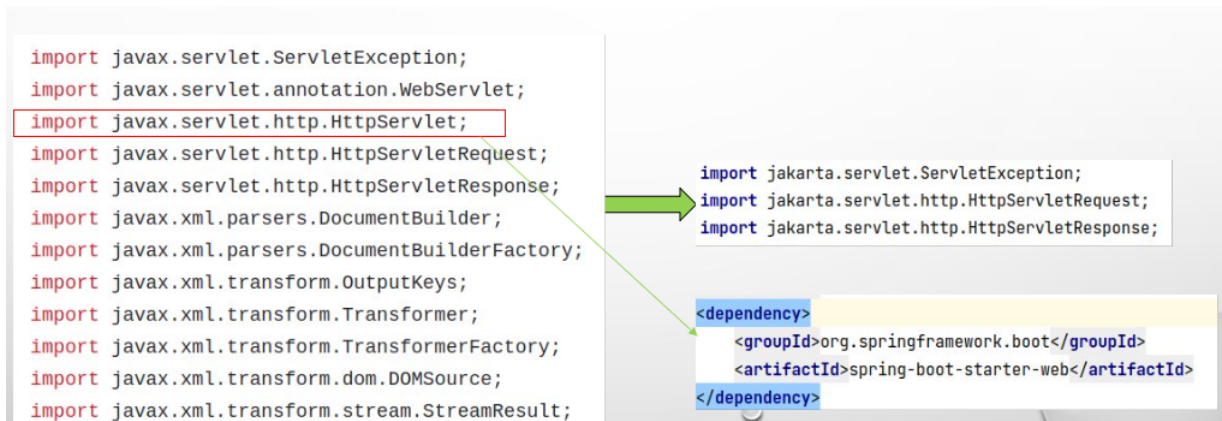


Figure 4.1: Javax libraries removed

The servlet functionalities were provided by spring boot dispatcher servlet and all the logic to handle the endpoints was migrated to controller, below diagram shows steps in how MVC pattern works.

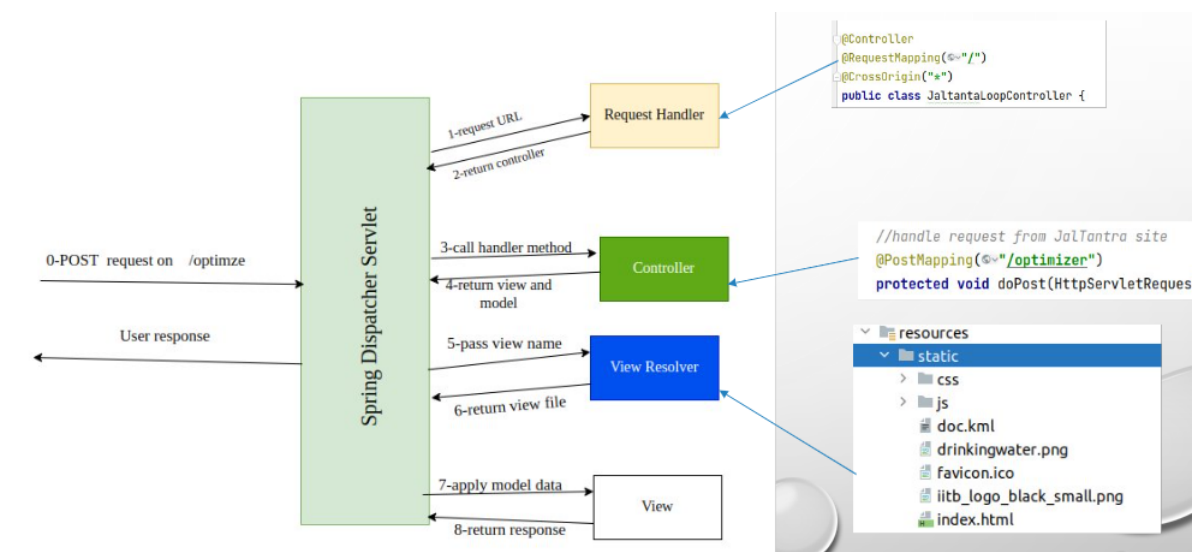


Figure 4.2: Spring MVC

- The dispatcher servlet receives the request, all requests to the web application is intercepted by dispatcher servlet, whose implementation is provided by spring starter web.
- Dispatcher servlet sends the request to request handler, in request handler all the controller are mapped, it returns the appropriate controller name to the servlet.

- Dispatcher servlet sends the request to controller and controller method to handle the request.
- Controller method on which request is mapped to sends the model data and view name back to servlet(i.e the data to be filled and view name).
- View and model data is sent to view resolver from the dispatcher servlet, and view resolver returns the view name and location.
- Finally the view with populated data is sent back as response to the client.

Below are advantages of MVC architecture over web-servlets:

- Spring MVC abstracts away the complexities of direct servlet handling, making the development process more straightforward and intuitive.Utilizes annotations (e.g., Controller, RequestMapping) to reduce boilerplate code and streamline request handling.
- Model-View-Controller (MVC) pattern provides a clear separation of concerns by dividing the application into three interconnected components: Model, View, and Controller.
- Decouples view implementation from the controller logic through configurable view resolver (e.g., ClassLoaderTemplateResolver).Custom configurations can be coded as required by extending WebMvcConfigurer.Supports various view technologies, including JSP, Thymeleaf, HTML.
- Offers fine-grained control over URL mappings using @RequestMapping, allowing more readable and manageable routing compared to servlets.Path Variables and Request Parameters simplifies the extraction and handling of request parameters and path variables, reducing manual parsing and casting.
- Provides built-in support for binding request parameters to Java objects and automatic validation using @Valid and other validation annotations.Automatically binds form data to POJOs (Plain Old Java Objects) and supports complex form handling scenarios.
- Simplifies the creation of RESTful web services with convenient annotations like RestController and.ResponseBody.Automatically handles content negotiation, allowing the same endpoint to serve different content types (e.g., JSON, XML). This feature is currently not used in application, but makes it easy to extend in future.

Description of some annotations used to simplify the migration to MVC architecture.

- `@Controller`
 - Purpose: Marks a class as a Spring MVC controller, indicating that it is a web controller responsible for handling HTTP requests.
 - Function: Simplifies the definition of controller classes by automatically registering them as Spring beans and managing the mapping of web requests to handler methods.
 - Benefit: Reduces boilerplate code and improves readability by clearly defining which classes handle web requests.
- `@RequestMapping`
 - Purpose: Maps web requests to specific handler methods within a controller.
 - Function: Allows fine-grained control over the routing of requests based on URL patterns, HTTP methods, and other request parameters.
 - Benefit: Enhances flexibility and readability by allowing developers to specify the exact conditions under which a method should handle a request.
- `@CrossOrigin`
 - Purpose: Enables Cross-Origin Resource Sharing (CORS) for a specific method or class, allowing web applications to access resources from different origins.
 - Function: Configures CORS policies, specifying which domains are permitted to make requests and what methods are allowed.
 - Benefit: Simplifies the setup of CORS, enhancing the security and accessibility of web applications by controlling cross-origin interactions.

Below is an illustration of the MVC advantage over web-servlet.

```

@WebServlet(urlPatterns = {"/hello", "/world"})
public class MultiActionServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String path = request.getServletPath();
        response.setContentType("text/html");

        if ("/hello".equals(path)) {
            response.getWriter().println("<h1>Hello, GET!</h1>");
        } else if ("/world".equals(path)) {
            response.getWriter().println("<h1>World, GET!</h1>");
        } else {
            response.getWriter().println("<h1>Unknown Path</h1>");
        }
    }
}

```

Figure 4.3: Inflexible coding style in web-servlet, all if conditions need to be handled in same method

```

<servlet>
  <servlet-name>MultiActionServlet</servlet-name>
  <servlet-class>com.example.MultiActionServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>MultiActionServlet</servlet-name>
  <url-pattern>/hello</url-pattern>
  <url-pattern>/world</url-pattern>
</servlet-mapping>

```

Figure 4.4: Extra boiler plate required in web.xml to configure the servlet

```

@Controller
public class HelloWorldController {

    @GetMapping("/hello")
    public String sayHelloGet(Model model) {
        model.addAttribute("message", "Hello, GET!");
        return "hello";
    }

    @GetMapping("/world")
    public String sayWorldGet(Model model) {
        model.addAttribute("message", "World, GET!");
        return "world";
    }
}

```

Figure 4.5: Flexible decoupled coding in MVC

Hence MVC was adopted for the Jaltantra application, and made it more flexible and robust.

4.3 Making code-base modular

Making code-base modular has below advantages.

- **Better Code Organization:** A modular approach enforces a clear and logical structure to the code-base, making it easier to navigate and understand.
- **Improved Maintainability**
 - **Separation of Concerns:** Modular code separates different functionalities into distinct modules, making it easier to manage and understand.
 - **Easier Updates:** Changes in one module can be made independently without affecting other parts of the application, simplifying maintenance and reducing the risk of introducing bugs.
 - **Isolated Issues:** Debugging is easier as issues can be isolated within specific modules.
 - **Targeted Fixes:** Bugs can be addressed more quickly by focusing on the affected module rather than the entire code-base.
- **Component Reuse:** Modules can be reused across different parts of the application or even in different projects, saving development time and effort, many components can be reused in other versions of Jaltantra as well like branch version.
- **Scalability**
 - **Incremental Scaling:** Applications can be scaled more efficiently by focusing on the modules that require additional resources.
 - **Distributed Systems:** Modular code can be more easily adapted to micro-services architecture, allowing parts of the application to be deployed and scaled independently.
- **Flexibility and Adaptability**
 - **Easier Refactoring:** Modules can be refactored or replaced with minimal impact on the overall system.
 - **Technology Upgrades:** Individual modules can be updated or rewritten using different technologies without affecting other parts of the application.

Below are steps involved in breaking the code base into maintainable modules

- **Identifying Cohesive Units:** Analyze the existing code-base to identify logical units of functionality that can be encapsulated into separate classes or modules. This could involve grouping related methods, data structures, or business logic together.
- **Separation of Concerns:** Aim to separate concerns within the code-base, adhering to principles such as Single Responsibility Principle (SRP) and Separation of Concerns (SoC). Each class or module should have a clear, focused purpose.
- **Encapsulation and Abstraction:** Encapsulate related functionality into classes with well-defined interfaces. This promotes abstraction and hides implementation details, making the code-base easier to understand and maintain.
- **Refactoring:** Refactor existing code to extract cohesive units of functionality into separate classes or modules. This may involve creating new classes, moving methods or fields, and updating dependencies between classes.
- **Testing:** Ensure that each modular class is thoroughly tested in isolation to verify its correctness and functionality. Unit tests, integration tests, and end-to-end tests can help validate the behavior of individual modules and their interactions.

4.3.1 Breaking down code into modules-Example

Below diagrams illustrate how `OptimizerServlet` class was broken down into more modular design.

OptimizerServlet	
doGet(HttpServletRequest, HttpServletResponse)	void
performNonOptimizationRelatedAction(HttpServletRequest, HttpServletResponse, String)	void
uploadExcelOutputFile(HttpServletRequest, HttpServletResponse)	void
logd(String)	void
doPost(HttpServletRequest, HttpServletResponse)	void
uploadMapSnapshotFile(HttpServletRequest, HttpServletResponse)	void
uploadXmlInputFile(HttpServletRequest, HttpServletResponse)	void
uploadExcelInputFile(HttpServletRequest, HttpServletResponse)	void
loge(String)	void
compareVersion(String, String)	int
logi(String)	void
getLinearElevation(int, int, double, double, NodeStruct[])	double
logw(String)	void
performOptimization(HttpServletRequest, HttpServletResponse)	void
uploadEpanetOutputFile(HttpServletRequest, HttpServletResponse)	void

Figure 4.6: Non modular class

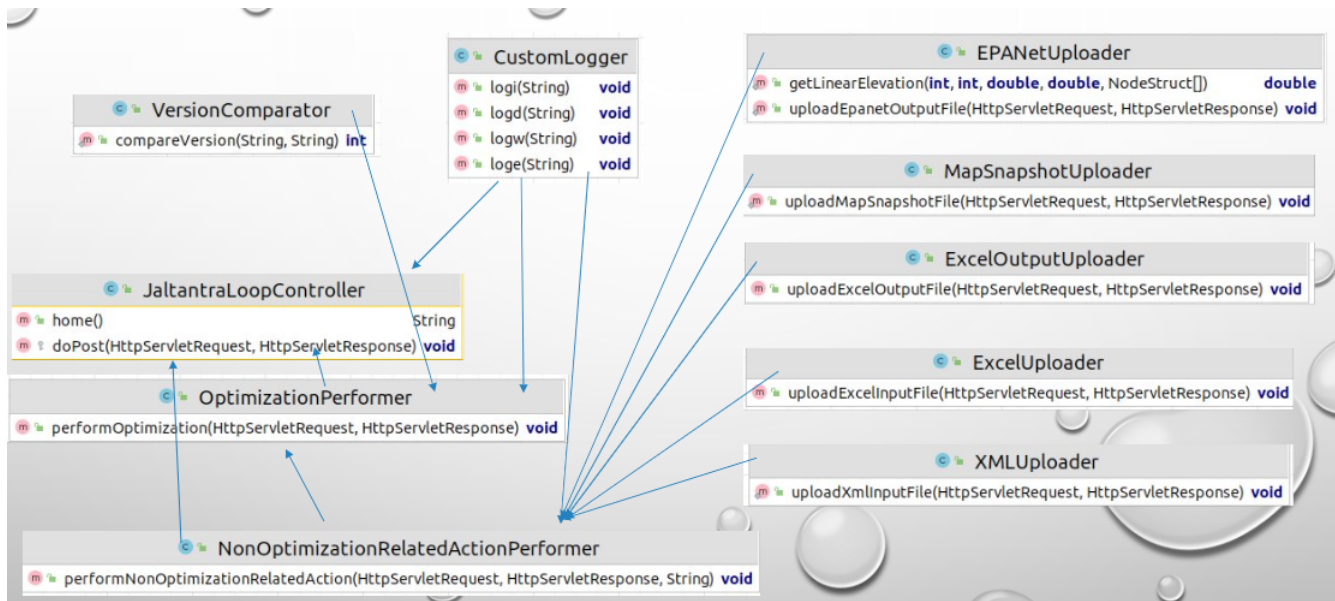


Figure 4.7: Modular class design

4.3.2 Wiring classes at runtime

Once the code-base is broken down into modular classes, these classes need to be wired together to form the application's runtime structure. This involves managing dependencies between classes and ensuring that the correct instances are created and used at runtime, this however is cumbersome process, due to below issues.

- **Manual Dependency Management:** Developers must manually create and manage dependencies, often using explicit constructor calls or setters.
- **Boilerplate Code:** Increases the amount of boilerplate code as developers have to write repetitive code for creating and wiring dependencies.
- **Hard-Coded Dependencies:** Dependencies are often hard-coded, leading to tightly coupled components and reduced flexibility.
- **Complex Initialization:** Initializing objects with dependencies can become complex and error-prone, especially as the number of dependencies grows.
- **Difficult Testing:** Unit testing becomes more challenging as it requires manual creation and injection of mock dependencies.

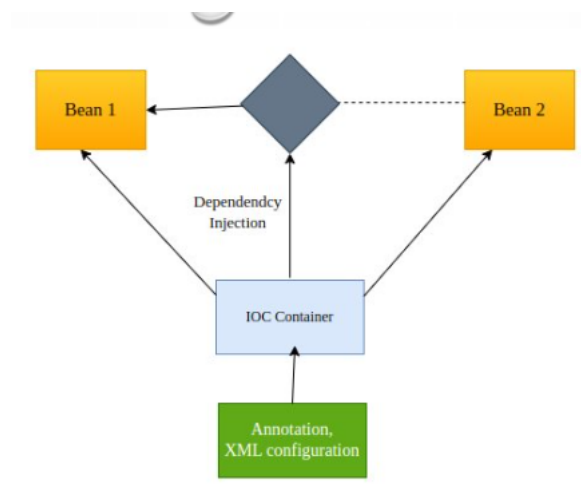


Figure 4.8: Dependency Injection

- **Dependency Injection (DI):** Dependency injection is a design pattern used to manage dependencies between classes. It involves injecting dependencies into a class rather than having the class create its dependencies directly. This promotes loose coupling and facilitates easier testing and maintenance.
- **Spring Boot and Dependency Injection:** Spring Boot provides built-in support for dependency injection through annotations such as `@Autowired`, `@Bean`, `@Component`, `@ComponentScan`, etc. These annotations allow to declare dependencies, specify how they should be instantiated, and wire them together automatically at runtime.

- **Annotation-Based Configuration:** Spring Boot leverages annotation-based configuration to wire classes together. By annotating classes with `@Component`, `@Service`, `@Repository`, etc., Spring Boot automatically detects and manages these components, injecting dependencies where needed.
- **Configuration Classes:** In addition to annotations, Spring Boot allows to define configuration classes where you can explicitly configure bean definitions and dependencies using Java code. This gives fine-grained control over how classes are wired together.
- **Application-Context:** Spring Boot's `ApplicationContext` manages the life-cycle of beans and their dependencies. It scans the class-path for annotated components, creates instances of these components, and resolves their dependencies, ensuring that the application's runtime structure is correctly configured.

4.3.3 Benefits of Spring boot's dependency injection

- **Promotes Loose Coupling:** Dependency injection promotes loose coupling between classes by removing direct dependencies and allowing dependencies to be injected at runtime. This makes the code-base more modular, flexible, and easier to maintain.
- **Simplifies Configuration:** Spring Boot's annotation-based configuration simplifies the configuration of dependencies by eliminating the need for XML configuration files. Dependencies can be declared and wired together using annotations, reducing boilerplate code and configuration overhead.
- **Facilitates Testing:** Dependency injection makes it easier to test individual components in isolation by allowing dependencies to be mocked or replaced with test doubles. This facilitates unit testing, integration testing, and mocking of external dependencies.
- **Enables Runtime Flexibility:** Dependency injection enables runtime flexibility by allowing dependencies to be swapped or configured differently based on the environment or configuration. This makes it easier to configure the application for different deployment scenarios without modifying the code-base.

Illustration-Spring boot's dependency injection

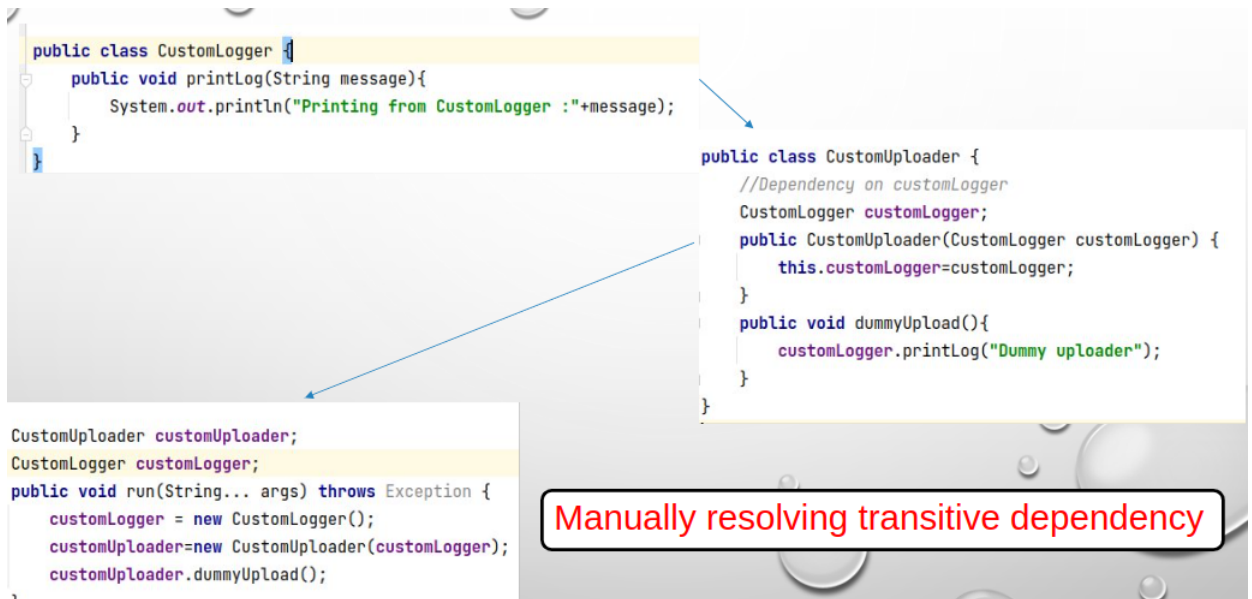


Figure 4.9: Dependency without annotation

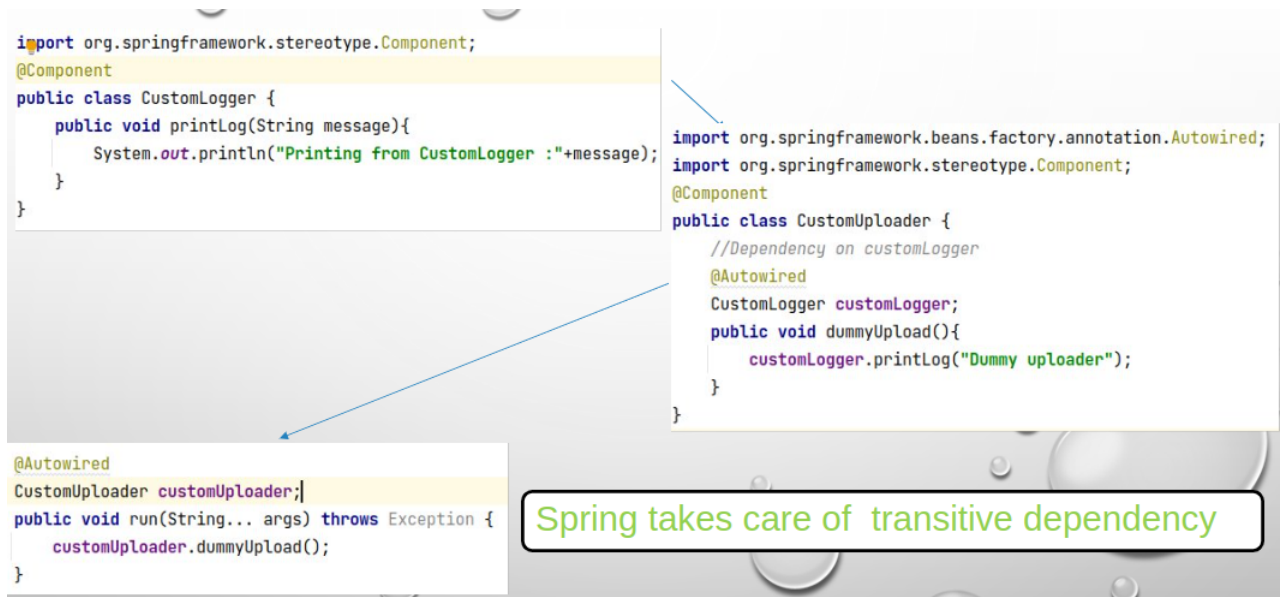


Figure 4.10: Dependency with annotation

4.4 Configuration management

4.4.1 Challenge

Making changes to configuration requires changes in multiple code files across code-base. Configurations change depending on environment eg. Deployment, development etc.

Configurations include:

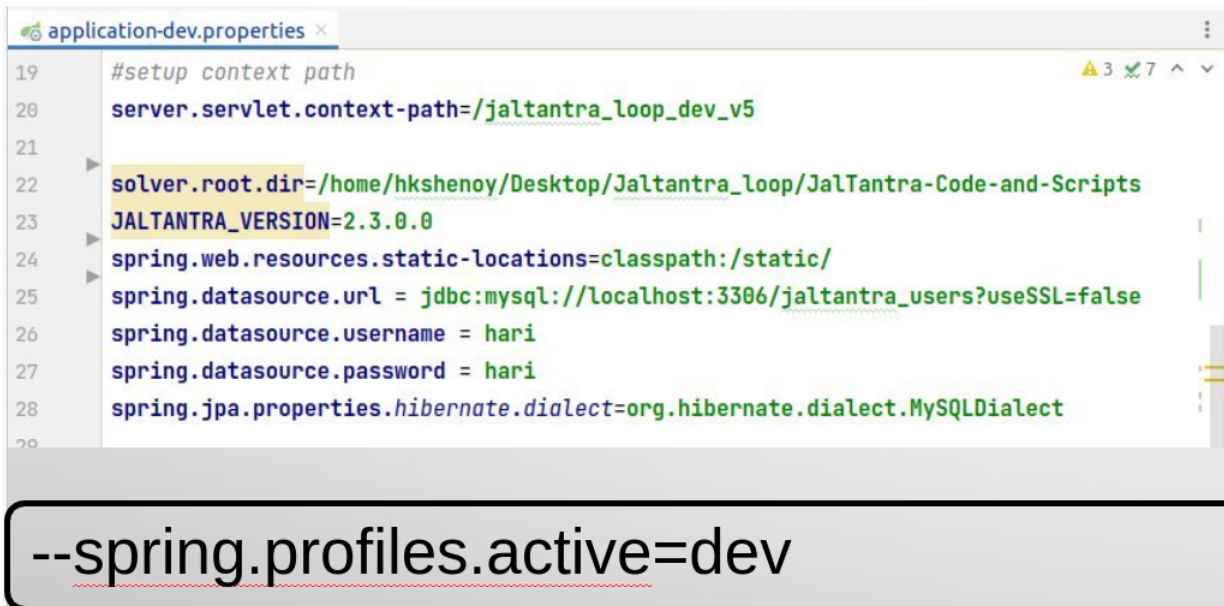
- Solver directory path: path for Jaltantra python and shell scripts which form the backend system for invoking the solvers (Non-linear solvers like Knitro, Baron etc) to solve the network file.
- Application context: unique path for the end points, to identify the requests coming to Jaltantra server and intercept them and send to the application.
- Ports: Ports on which application is listening, may be required to change depending on already occupied ports in system.
- Database URL's: support for any database connections URL's required for application to work might change on different physical machines, hence is also configurable
- Passwords: Passwords related to database logins, mail logins, API keys etc, also might be need to change depending on the physical systems.

4.4.2 Implementation

Spring Boot profiles and properties files are fundamental components that enables to configure and customize Spring Boot application for different environments and use cases.

Spring Boot profiles are a way to segregate configurations for different environments or runtime conditions within application. By using profiles, we can define sets of configurations that apply only when the application is running in a specific profile., which can be invoked simply by argument `spring.profiles.active=dev` when application is run.

Properties files are used in Spring Boot to configure application properties such as solver directory, database settings, server ports, logging levels, etc. These files can be in either `.properties` or `.yaml` format and are typically located in the `src/main/resources` directory of project.



```
19 #setup context path
20 server.servlet.context-path=/jaltantra_loop_dev_v5
21
22 solver.root.dir=/home/hkshenoy/Desktop/Jaltantra_loop/JalTantra-Code-and-Scripts
23 JALTANTRA_VERSION=2.3.0.0
24 spring.web.resources.static-locations=classpath:/static/
25 spring.datasource.url = jdbc:mysql://localhost:3306/jaltantra_users?useSSL=false
26 spring.datasource.username = hari
27 spring.datasource.password = hari
28 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
29
```

--spring.profiles.active=dev

Figure 4.11: Application properties file- Development environment



```
12 #setup context path
13 server.servlet.context-path=/jaltantra_loop_dev_v6
14 solver.root.dir=/home/deploy/Jaltantra_v2_3_0_0/JalTantra-Code-and-Scripts
15 JALTANTRA_VERSION=2.3.0.0
16 spring.web.resources.static-locations=classpath:/static/
17 spring.datasource.url = jdbc:mysql://localhost:3306/jaltantra_users?useSSL=false
18 spring.datasource.username = deploy
19 spring.datasource.password = deploy
20 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
21
```

--spring.profiles.active=deploy

Figure 4.12: Application properties file- Deploy environment

below briefly describes internal working of the implementation.


```

@Component
public class OptimizationPerformer {

    @Autowired
    NonOptimizationRelatedActionPerformer nonOptimizationRelatedActionPerformer;

    @Value("${JALTANTRA_VERSION}")
    private String version;

    @Value("${solver.root.dir}")
    private String solverRootDir;
}

```

Figure 4.13: @Value annotation

- Spring boot provides ‘application.properties’ file which ‘injects’ value at run-time to respective variables in project.
- Value is injected in code by using @Value annotation at suitable places in code, the @Value annotation is used to inject values from properties files, environment variables, or other Spring beans into fields in Spring-managed beans (components, services, controllers, etc.).
- It provides a convenient way to externalize configuration and inject dynamic values into application without the need for explicit property file parsing or environment variable retrieval.

Using Thymeleaf functionalities as shown in below figure, values from properties file were injected at runtime into Java-script based front-end as well.

```

<script th:inline="javascript">
    var contextPath='[[${contextPath}]]';
    contextPath=contextPath.replace(/"/g, '');
    console.log(contextPath);

    const optimizerUrl=contextPath + '/optimizer';
    console.log(optimizerUrl);

    var currentVersion='[[${version}]]';
    currentVersion=currentVersion.replace(/"/g, '');
    console.log(currentVersion)
</script>

```

Figure 4.14: Values injected using Thymeleaf template engine

4.5 Removing external tomcat dependency

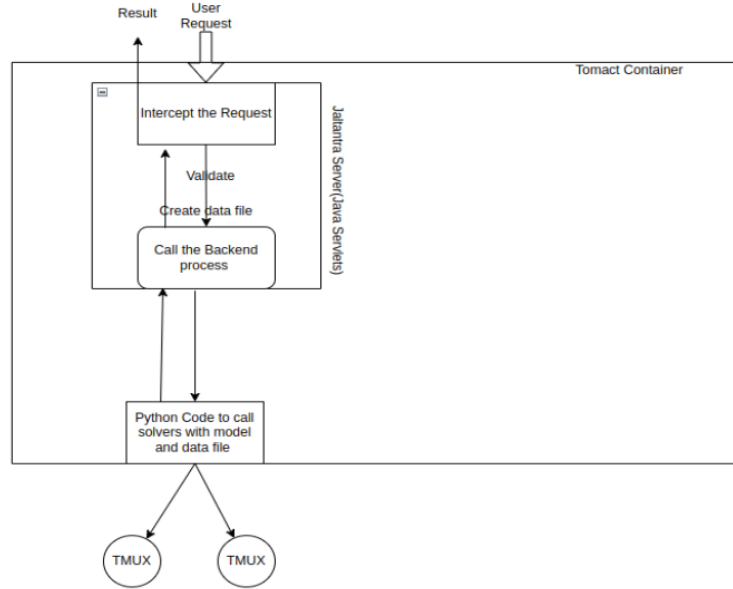


Figure 4.15: External Tomcat dependency

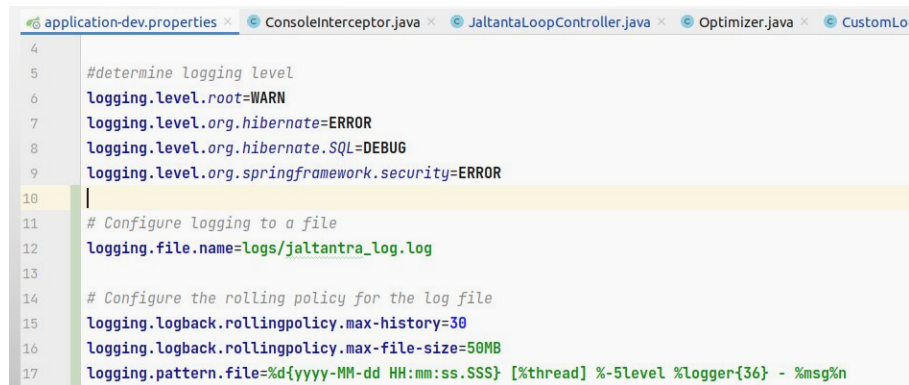
Jaltantra loop web-app is deployed as WAR file on Tomcat server running on the machine, tomcat is responsible for calling back-end python process and also the logs for application are populated by the tomcat.spring starter web dependency provides the jar file for embedded tomcat. Below are the advantages of the embedded server architecture.

- **Resource Efficiency:** Embedded servers are designed to be lightweight and consume fewer system resources compared to standalone servers. This makes them ideal for applications running on resource-constrained devices or in environments where efficiency is crucial. By eliminating network overhead and communication delays, embedded servers can provide faster response times and better scalability.
- **Simplified Deployment:** An embedded server integrates the server, such as Tomcat, directly into the application's runtime environment, allowing the server to run within the same process as the application. This approach simplifies deployment, as the application and server are bundled together and launched as a single executable unit, streamlining configuration and ensuring consistency across different environments.

- **Portability:** Embedded servers can be packaged with the application code, making the entire application self-contained and portable. This allows developers to deploy applications across different environments without worrying about compatibility issues or dependencies on external server configurations.
- **Reduced Maintenance Overhead:** Since embedded servers are integrated directly into the application, there is less maintenance overhead compared to standalone servers. Developers can easily update and manage the server component along with the application code, without the need for separate server administration tasks.
- **Customization and Control:** Embedded servers allow developers to customize and control the server configuration directly within the application code. This provides greater flexibility and enables developers to optimize the server settings for specific application requirements.
- **Security:** Embedded servers can enhance security by reducing the attack surface and minimizing exposure to external threats. Since the server is embedded within the application, there are fewer entry points for potential attackers, making it harder to exploit vulnerabilities.

4.6 Granular logging

Control and flexibility over logging is important in bug fixing and development process, spring boot provides logging libraries and also configurable properties like file size, location, name, levels (i.e. amount of information regarding events at runtime) etc. to customize the logging. Also console printed statements are redirected to the logging file, by coding customized classes as illustrated below.



```

4
5 #determine logging level
6 logging.level.root=WARN
7 logging.level.org.hibernate=ERROR
8 logging.level.org.hibernate.SQL=DEBUG
9 logging.level.org.springframework.security=ERROR
10
11 # Configure logging to a file
12 logging.file.name=logs/jaltantra_log.log
13
14 # Configure the rolling policy for the log file
15 logging.logback.rollingpolicy.max-history=30
16 logging.logback.rollingpolicy.max-file-size=50MB
17 logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n

```

Figure 4.16: Logging properties



```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import jakarta.annotation.PostConstruct;
import java.io.PrintStream;

@Component
public class ConsoleInterceptor {

    private static final Logger logger = LoggerFactory.getLogger(ConsoleInterceptor.class);

    @PostConstruct
    public void init() {
        // Redirect System.out to the logging system
        System.setOut(new PrintStream(System.out) {
            @Override
            public void println(String message) { logger.info(message); }
        });

        // Redirect System.err to the logging system
        System.setErr(new PrintStream(System.err) {
            @Override
            public void println(String message) { logger.error(message); }
        });
    }
}

```

Figure 4.17: Console Interceptor

Further the logging classes like interceptor class are completely decoupled from main application logic hence, they can be customized as required without any effect on other parts of code.

4.7 Build automation

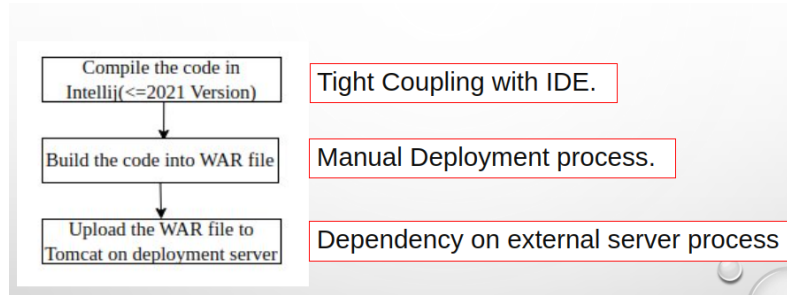


Figure 4.18: Manual build process

Jalntantra had a manual build process(as described in above figure),https://github.com/jaltantra/Jaltantra_loop/wiki this hindered the development, bug-fixing as well as developer experience and made development work cumbersome on the Jaltantra project. Automation was done with help of Spring boot provided maven plugins and maven wrapper.

4.7.1 Maven

Maven [2] is a powerful build automation tool and project management tool for Java-based projects. It provides a standard way to define project structures, manage dependencies, and automate the build process. Maven is commonly used as the build automation tool for Spring Boot projects due to its seamless integration with the Spring ecosystem and its robust dependency management capabilities. The pom.xml file is a core component of Maven projects. It stands for "Project Object Model" and serves as the configuration file for Maven. Below are s of pom.xml. The Maven Wrapper is a tool that builds Maven project with a specific version of Maven, regardless of the version installed in local machine. It ensures that the build is reproducible and consistent across different environments, mvnw and mvnw.cmd are the wrapper scripts for Unix-based systems and Windows, respectively. They are used to execute Maven commands and these scripts are available in spring boot projects.

- Project Structure and Dependency Management: The pom.xml serves as the centerpiece of a Maven project, defining its structure, dependencies, and build configuration. In a Spring Framework project, the pom.xml manages dependencies on Spring libraries, ensuring that the project has access to the required Spring modules and related dependencies.

- **Consistency and Reproducible:** Maven provides a standardized project structure and build life-cycle, ensuring consistency across different projects and facilitating collaboration among team members. With Maven, developers can easily reproduce the build environment on different machines or CI/CD pipelines, reducing the likelihood of environment-related issues and ensuring reliable builds.
- **Dependency Management and Resolution:** Maven simplifies dependency management by automatically resolving and downloading dependencies from remote repositories based on declarations in the pom.xml. For Spring projects, Maven streamlines the integration of various Spring modules, libraries, and third-party dependencies, ensuring smooth development and deployment workflows.
- Maven resolves these dependencies by downloading the required packages and jars from maven central repository, for efficiency it caches these dependency files in local machine.

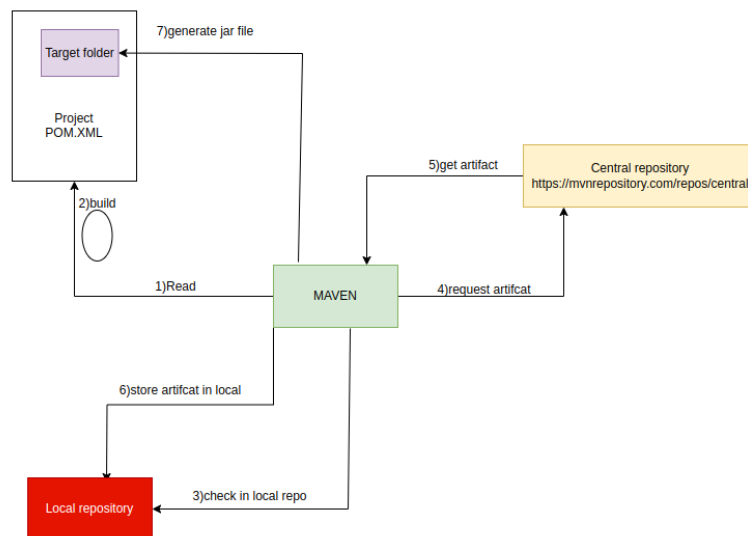


Figure 4.19: Maven working

4.8 Migration workflow

During the migration process from a Java Servlet-based application to Spring Boot, significant effort was invested in code refactoring and replacing outdated libraries

with modern alternatives. This initiative was undertaken to enhance the application's architecture, performance, maintainability, and alignment with industry best practices. Below are the detailed workflow done to achieve the migration.

- Aligning code with MVC architecture-servlets were migrated as controllers , and static pages served as view pages and appropriate configurations for view resolution.
- Mapping of servlet request handlers by using spring boot annotations. all the controllers were annotated with appropriate end points and request method annotations to migrate the servlet functionality to MVC architecture.
- Updating the outdated libraries: below steps were followed for outdated library replacement
 - Evaluation of Existing Libraries: An exhaustive analysis of the existing libraries and dependencies was conducted to assess their compatibility, performance, and relevance in the Spring Boot ecosystem.
 - Identification of Deprecated or Unsupported Libraries: Deprecated or unsupported libraries were identified and replaced with modern alternatives that are actively maintained and aligned with the project's requirements.
 - Integration of Spring Ecosystem: Libraries from the Spring ecosystem, such as Spring Data, Spring Security, and Spring MVC, were introduced to leverage their robust features and seamless integration with Spring Boot.
 - Compliance with Industry Standards: Libraries that comply with industry standards, such as RESTful API specifications, data validation frameworks, and logging utilities, were integrated to ensure adherence to best practices.
- Breaking down classes into modular and manageable classes:
 - Identification of Legacy Code: The first step involved identifying legacy code segments, including redundant, deprecated, or poorly structured code.
 - Improving Code Structure: Refactoring techniques such as method extraction, class restructuring, and package reorganization were applied to improve code readability, maintainability, and scalability.

- Elimination of Code Smells: Code smells such as duplication, long methods, and excessive complexity were addressed to enhance code quality.
- Introduction of Design Patterns: Where appropriate, common design patterns such as Singleton, and Dependency Injection were implemented to promote modularization and decoupling of components, as illustrated.
- Re-wiring them with help of spring boot annotations-used spring boot annotations to simplify transitive dependency and make code structure cleaner,
- Extracting configurations to properties file-Variious variables which were dependent on runtime environment,paths, context etc were identified in front end and middle ware and these properties were injected at runtime from application.properties file.
- Setting up logging and build automation-Utilized spring boot provided plugins and properties to setup appropriate customizable logging and build system.
- Iterative testing at each step to ensure functionalities are intact:
 - Unit Testing: Comprehensive unit tests were developed and executed to validate the behavior and functionality of refactored code segments and newly integrated libraries.
 - Integration Testing: Integration tests were conducted to verify the seamless interaction between components and to ensure the stability and correctness of the overall system, end to end results for inputs were analyzed from migrated system and current system.
 - Regression Testing: Regression tests were performed to validate that the migration process did not introduce unintended side effects or regressions in the application's behavior.

4.9 Way-ahead

Jaltantra web-application was successfully migrated to spring boot, the chapter described various features of spring boot used in migration process, in next chapter enhancements to the application made using spring boot will be discussed.

Chapter 5

Feature enhancement

This chapter describes various features added above existing jaltantra code-base in addition to migration. Features like login security with email verification, deployment automation and automated monitoring have been added.

5.1 User management and security

User management is important aspect of keeping track of application users. Ideal user management systems should provide login, registration, user sessions details persistence, user request details persistence. For implementing these features spring security and spring data JPA for persistence will be used, these packages help in implementing these features in comprehensive Spring Security is a comprehensive security framework that provides robust security features to protect Java applications from various security threats. It protected all type of application architecture like a web application, REST API, or micro-services architecture, Spring Security offers the flexibility, scalability, and reliability needed to implement secure authentication and authorization mechanisms.

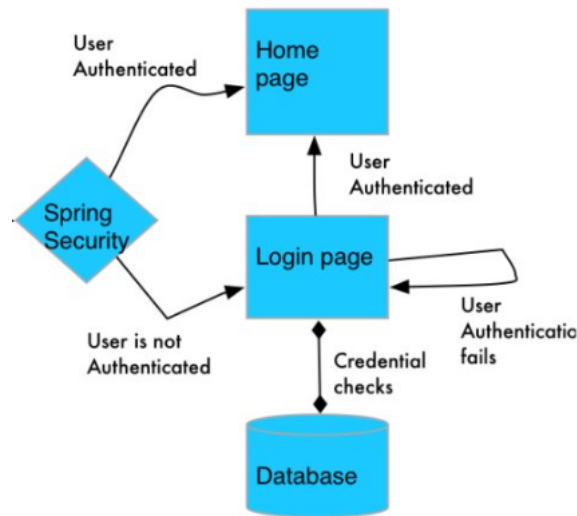


Figure 5.1: Securing end points

Below are some features of spring security

- **Authentication:** Spring Security provides various authentication mechanisms, including form-based authentication, HTTP Basic/Digest authentication, LDAP authentication, OAuth, and OpenID authentication. It supports integration with various authentication providers, such as JDBC, LDAP, in-memory, and custom authentication providers.
- **Authorization:** Spring Security enables fine-grained access control through authorization mechanisms. It allows developer to define access control rules based on user roles, permissions, and expressions. You can configure method-level security using annotations or XML configuration, as well as URL-based security using Ant-style patterns.
- **Session Management:** Spring Security offers session management capabilities to manage user sessions securely. It supports features such as session fixation protection, concurrent session control, session timeout configuration, and session registry for tracking active sessions.
- **Web Security:** Spring Security provides protection against common web vulnerabilities, such as CSRF (Cross-Site Request Forgery), XSS (Cross-Site Scripting), and Clickjacking attacks. It offers built-in mechanisms to mitigate these vulnerabilities, including CSRF tokens, Content Security Policy (CSP), and XSS protection headers.

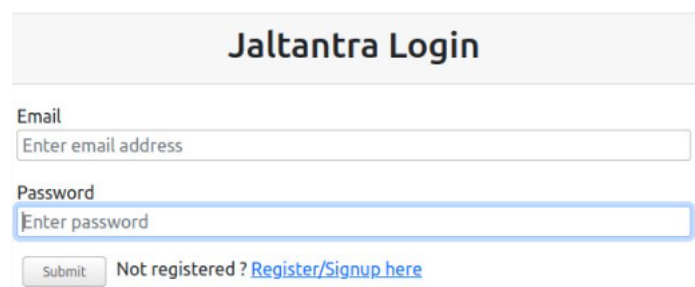
- **Integration with Spring Framework:** Spring Security seamlessly integrates with other components of the Spring ecosystem, such as Spring MVC, Spring Boot, and Spring Data. It provides configuration options for integrating security features into Spring applications, making it easy to secure both web and non-web components.
- **Customization and Extension:** Spring Security is highly extensible, allowing to tailor security configurations to fit application's specific requirements. It allows to create custom authentication providers, access control rules, authentication filters, and error handling mechanisms to adapt Spring Security to application's security needs.
- **Community Support and Documentation:** Spring Security benefits from a large and active community of developers and contributors. It offers extensive documentation, tutorials, and online resources to help you get started with security configuration and troubleshooting.

5.2 Implementing user management

This section describes the detailed workflow to bring spring security to action in the web app

5.2.1 Login and register Pages

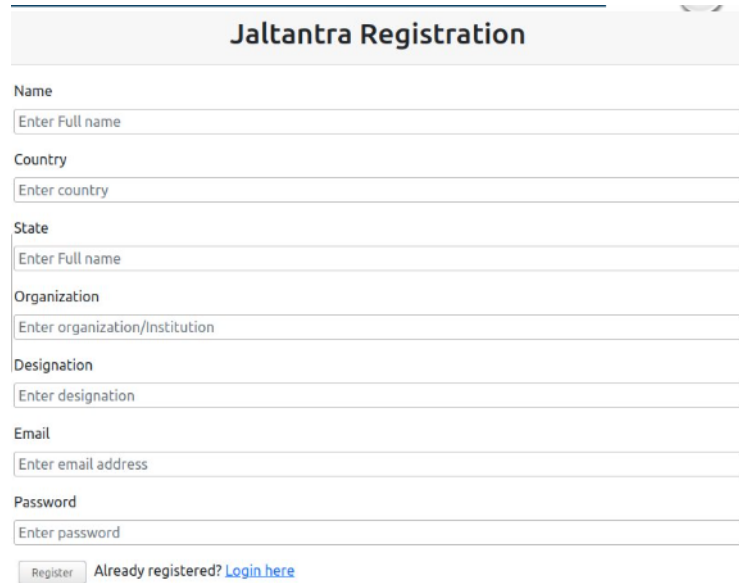
- **Login Page:** Created a web page where users can enter their credentials (username and password) to authenticate themselves. This page includes form fields for username and password, as well as a submit button to submit the login credentials. Below is the simplistic design of login page for Jaltantra. The Login page uses client side SHA-512 hashing for secrecy of password entered.



The image shows a login form titled "Jaltantra Login". It contains two input fields: "Email" with the placeholder text "Enter email address" and "Password" with the placeholder text "Enter password". Below the password field is a "Submit" button and a link that says "Not registered ? [Register/Signup here](#)".

Figure 5.2: Jaltantra Login

- Register Page: Created a web page where new users can register by providing their details, such as name, password, email address, organization details, country etc. This page includes form fields for user registration details, along with validations to ensure data integrity. The password entered at registration is hashed at client side using SHA-512 and stored in server database and used to validate password at login.



Jaltantra Registration

Name
Enter Full name

Country
Enter country

State
Enter Full name

Organization
Enter organization/Institution

Designation
Enter designation

Email
Enter email address

Password
Enter password

Register Already registered? [Login here](#)

Figure 5.3: Jaltantra Registration

5.2.2 Defining database schema

- User Model: Define a user model class to represent user details, such as username, password, email, etc. This class may also include fields for user roles.
- Role Model: Define a role model class to represent user roles, such as USER, ADMIN, etc.
- Authentication Service: Implement an authentication service to handle user authentication and authorization. This service verifies user credentials during login and assigns roles to authenticated users.
- User Repository: Create a repository interface to interact with the database and perform CRUD operations on user entities.

5.2.3 Persisting user details

The underlying database used in MYSQL [8] and below is the details of schema used to persist the user details.

- Database Schema: Design a database schema to store user details, including tables for users, roles, and user-role mappings. Use appropriate data types and constraints to ensure data integrity and security.
- Database Configuration: Configure the application to connect to the database using JDBC or an ORM framework like Hibernate, configured data source properties, connection pool settings, etc. in the application configuration.
- Data Access Layer: Implemented data access logic to interact with the database, including CRUD operations, query execution, and transaction management.

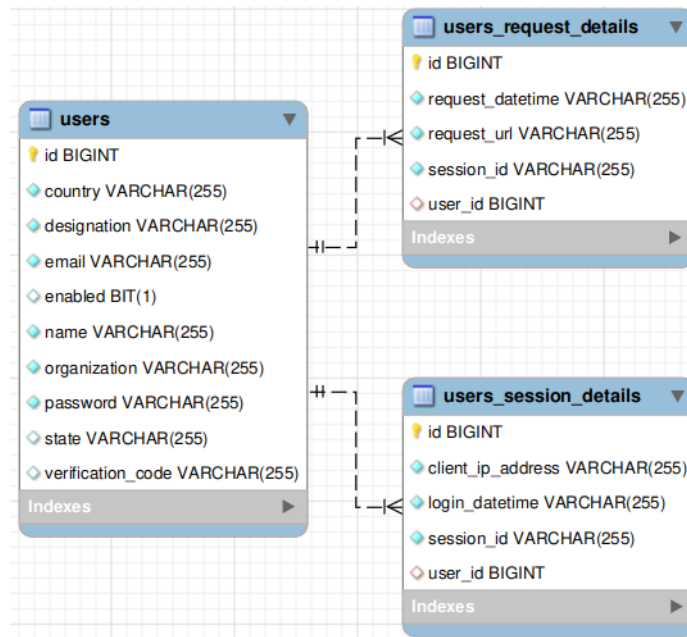


Figure 5.4: Database Schema

Schema description:

- Users table:
 - id: The id field in the users table is a bigint type that serves as the primary key. It uniquely identifies each user in the database and is automatically incremented with each new record, ensuring a unique ID for every user.

- country: The country field is a varchar(255) type that stores the country of the user. This field does not allow null values, ensuring that the country information is always provided for each user.
 - designation: The designation field is a varchar(255) type that records the user’s job title or designation. Like the country field, it also does not allow null values, ensuring every user has a designation listed.
 - email: The email field is a varchar(255) type that must be unique for each user, preventing any duplicate email entries. This field is crucial for user identification and communication, and it does not allow null values.
 - enabled: The enabled field is a bit(1) type that indicates whether the user account is active or not. This field can be null, allowing for accounts that have not yet been activated or have been deactivated.
 - name: The name field is a varchar(255) type that stores the full name of the user. It is a required field, ensuring that each user has a name recorded in the database.
 - organization: The organization field is a varchar(255) type that records the organization to which the user belongs. This field does not allow null values, making it mandatory for every user to be associated with an organization.
 - password: The password field is a varchar(255) type that stores the user’s password, typically in a hashed format for security purposes. It is a required field, ensuring that every user has a password for account protection.
 - state: The state field is a varchar(255) type that optionally records the state or province of the user. This field can be null, allowing flexibility for users who may not need to provide state information.
 - verification_code: The verification_code field is a varchar(255) type used to store a code for verifying the user’s email or account. This field can be null, as it is only used during the registration or account recovery processes.
- users_session_details
 - id: The id field in the users_session_details table is a bigint type that serves as the primary key, uniquely identifying each session detail record.

It is automatically incremented, ensuring a unique identifier for each session.

- `client_ip_address`: The `client_ip_address` field is a `varchar(255)` type that stores the IP address of the client (user) during the session. This field does not allow null values, ensuring that the IP address is always recorded for each session.
- `login_datetime`: The `login_datetime` field is a `varchar(255)` type that records the date and time when the user logged in, stored as a string. This field does not allow null values, making it mandatory to track the login time for each session.
- `session_id`: The `session_id` field is a `varchar(255)` type that uniquely identifies each session. This field does not allow null values, ensuring that every session is uniquely tracked.
- `user_id`: The `user_id` field is a `bigint` type that serves as a foreign key referencing the `id` field in the `users` table, representing the user to whom the session belongs. This field can be null, allowing for sessions that may not be directly linked to a specific user.

- `users_request_details`

- `id`: The `id` field in the `users_request_details` table is a `bigint` type that serves as the primary key, uniquely identifying each request detail record. It is automatically incremented, ensuring a unique identifier for each request.
- `request_datetime`: The `request_datetime` field is a `varchar(255)` type that records the date and time when the request was made, stored as a string. This field does not allow null values, making it mandatory to track the time of each request.
- `request_url`: The `request_url` field is a `varchar(255)` type that stores the URL requested by the user during the session. This field does not allow null values, ensuring that the URL is always recorded for each request.
- `session_id`: The `session_id` field is a `varchar(255)` type that identifies the session during which the request was made. This field does not allow null values, ensuring that each request can be traced back to a specific session.

- user_id: The user_id field is a bigint type that serves as a foreign key referencing the id field in the users table, representing the user who made the request. This field can be null, allowing for requests that may not be directly linked to a specific user.

Having user database that tracks sessions, IP's, requests etc will help in automating using database triggers for any specific events, for e.g if any malicious users/ip etc can be disabled by triggers. Database triggers are special types of stored procedures that automatically execute or "trigger" in response to certain events on a particular table or view. These events are usually database operations such as INSERT, UPDATE, or DELETE. Triggers can be used to enforce rules, maintain data integrity, and automate system tasks. Below is a illustration of SQL trigger, based on user requests(this is not a live feature of application, if required as user base grows below can be used by developers).

```
CREATE TRIGGER check_user_activity
AFTER INSERT ON users_request_details
FOR EACH ROW
BEGIN
    DECLARE request_count INT;

    -- Count requests in the last 24 hours
    SELECT COUNT(*) INTO request_count
    FROM users_request_details
    WHERE user_id = NEW.user_id AND request_date_time >= NOW() - INTERVAL 1 DAY;

    -- If the user has more than 6 logins or 15 requests in the last 24 hours, disable the user
    IF request_count > 15 THEN
        UPDATE users
        SET enabled = 0
        WHERE id = NEW.user_id;
    END IF;
END;
```

Figure 5.5: Trigger example

```
mysql> select * from users_session_details;
+-----+-----+-----+-----+-----+
| id | client_ip_address | login_datetime | session_id | user_id |
+-----+-----+-----+-----+-----+
| 1 | 0:0:0:0:0:0:1 | 2024.06.05.17.55.47 | 95648DAEE680D4B16795C5D55A3032ED | 1 |
| 2 | 192.168.0.104 | 2024.06.05.18.32.37 | AC186F94153127390910E61B3EDB8A9D | 1 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Figure 5.6: User session data

```
mysql> select * from users_request_details;
```

id	request_datetime	request_url	session_id	user_id
1	2024.06.06.16.22.07	/jaltantra_loop_dev_v7/optimizer	9935DE680B0E3581F1B8812956D05D14	1
2	2024.06.06.16.22.09	/jaltantra_loop_dev_v7/optimizer	9935DE680B0E3581F1B8812956D05D14	1
3	2024.06.06.16.23.41	/jaltantra_loop_dev_v7/optimizer	9935DE680B0E3581F1B8812956D05D14	1
4	2024.06.06.16.24.27	/jaltantra_loop_dev_v7/branch/optimize	9935DE680B0E3581F1B8812956D05D14	1

Figure 5.7: Verification Email

5.2.4 Protecting URLs using spring security

- **Spring Security Configuration:** Created a Spring Security configuration class to define security rules and access control policies. Use annotations or Java configuration to specify URL patterns, authentication requirements, and role-based access control rules.
- **Authentication Provider:** Configure an authentication provider to authenticate users based on credentials stored in the database. Implement authentication logic to verify user credentials and authenticate users during login.
- **Authorization Provider:** Configure an authorization provider to enforce access control policies based on user roles. Define access control rules to restrict access to certain URLs or resources based on user roles.
- **Security Filters:** Configure security filters to intercept incoming requests and enforce security policies, such as authentication, authorization, CSRF protection, etc.
- **Login and Logout Handling:** Configure login and logout handlers to handle authentication and logout requests. Define login and logout URLs, success and failure handlers, redirection URLs, etc.

All the above functionalities are coded in a modular way ,i.e in easier to maintain and extend if required in future as a reponse to dynamic nature of security threats.

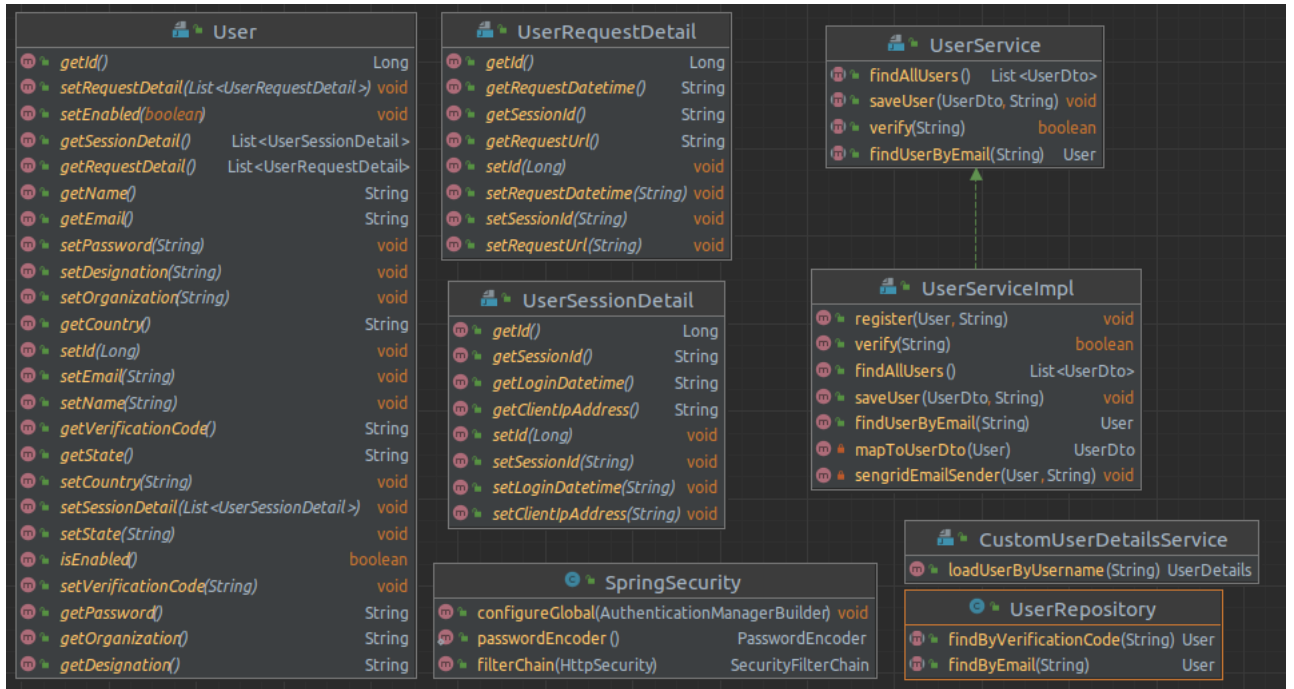


Figure 5.8: Security Classes

5.2.5 Email verification

This sub section describes Email verification feature in Jaltantra security flow. This feature is currently manual and

- Verification Email Sent to Registered Email Address
 - :When a user registers on the Jaltantra platform, a verification email is automatically sent to the email address provided during registration.
 - This email contains a unique verification link or code that the user needs to click on or enter to verify their email address.
 - The purpose of sending the verification email is to ensure that the user has access to the provided email address and to validate the ownership of the account.
- User Clicks on Verification Link
 - Upon receiving the verification email, the user clicks on the verification link provided in the email.

- Clicking on the link redirects the user to the verification page on the Jaltantra platform.
- The verification link typically contains a unique identifier or token that is associated with the user’s account and is used to verify the email address.
- Database Query for Verification Code
 - When the user is redirected to the verification page, the Jaltantra platform queries the database to retrieve the verification code associated with the user’s account.
 - This verification code is compared with the code provided in the verification link clicked by the user.
- Verification Code Match
 - If the verification code retrieved from the database matches the code provided in the verification link, the user’s account is marked as enabled.
 - This enables the user to log in and access the Jaltantra platform’s features and functionalities.
 - Setting the user account as enabled ensures that only users with verified email addresses can access the platform, enhancing security and trust.
- Ensuring Valid Email Addresses and Securing Endpoints
 - The verification process ensures that only users with valid and verified email addresses can register and use the Jaltantra platform.
 - By requiring users to verify their email addresses, the platform reduces the risk of fake or spam accounts and enhances the overall security of the platform.
 - Additionally, securing endpoints with email verification helps protect against unauthorized access and potential attacks, such as account takeover or unauthorized data access.

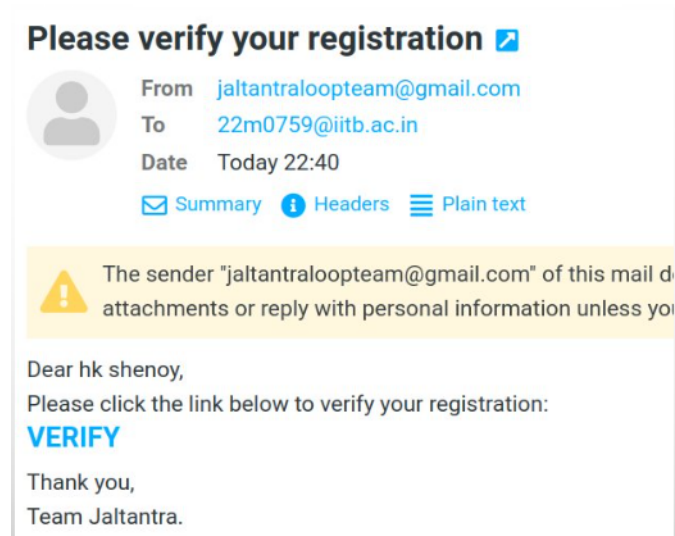


Figure 5.9: Verification Email

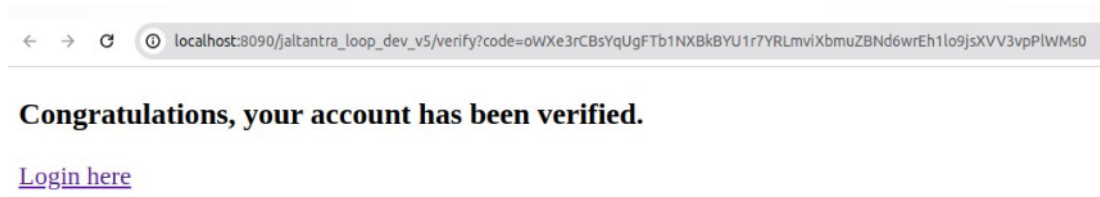


Figure 5.10: Verification successful

5.3 Deployment automation

This section describes automating the deployment process of a Spring Boot application into a 1 click process

5.3.1 Using Maven Wrappers and Bash Scripting

- Maven wrappers [2], combined with bash scripting, streamline the deployment process into a single click operation.
- Maven wrappers ensure that the necessary Maven version is available regardless of the environment, eliminating the need for Maven installation and version management.
- Bash scripting orchestrates the deployment steps, including building the application, packaging it into a JAR file, and transferring it to the deployment

server.

5.3.2 JSON configurable parameters

- Deployment parameters such as server details, environment settings, and application configurations are centralized and configurable via a single JSON file.
- This JSON file allows for easy customization and management of deployment settings without modifying the deployment script itself.
- Parameters such as server host, port, database credentials, logging levels, etc., can be specified in the JSON file and dynamically applied during deployment
- A script triggered from the local machine automates the transfer of the application's JAR file and its dependencies to the deployment server as mentioned in JSON file.
- This script uses secure file transfer protocols (e.g., SCP, SFTP) to ensure the safe and efficient transfer of files from the local development environment to the remote deployment server
- By automating the file transfer process, manual intervention is minimized, reducing the risk of errors and accelerating the deployment cycle.

By implementing above, the deployment process for the Jaltantra Spring Boot application becomes seamless, efficient, and easily customizable in a single click. It enables rapid and reliable deployment of new application versions, ensuring that updates and changes can be deployed with minimal effort and maximum consistency.

5.4 Automated monitoring

Jaltantra has been enhanced with automated monitoring. Deployed applications can run into issues, and to minimize impact of issues on user experience, automated monitoring is essential, as it can reduce the response time and efforts of developer to diagnose the issue and resolve it.

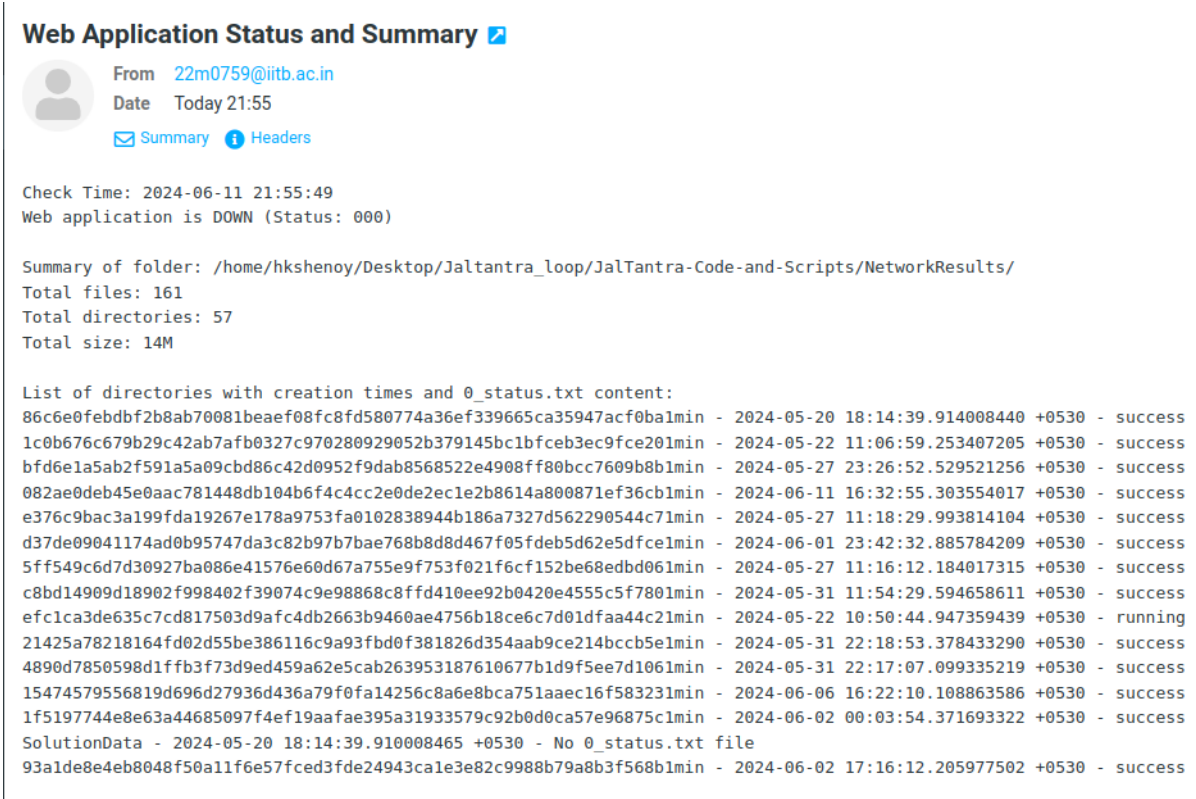


Figure 5.11: Application monitoring

Below are some issues which might occur in system which are a motivation for implementing automated monitoring.

5.4.1 Motivation

- Launcher Errors Due to Licensing Issues
 - Launcher errors due to licensing issues can disrupt the normal functioning of the application and impact user experience.
 - To address this, bash scripting is used to continuously monitor the status of endpoints associated with the application launcher.
 - By continuously checking for launcher errors in real-time, developers are promptly alerted to any licensing issues, enabling them to take immediate action to resolve them.
- Data Not Being Feasible to Solve

- Sometimes, data-related issues may arise within the application that are not immediately solvable or require further investigation.
- To keep developers informed about such issues, bash scripting is utilized to monitor the application's data integrity and feasibility continuously.
- By monitoring data-related issues in real-time, developers are notified promptly, allowing them to investigate and address the underlying causes efficiently.

5.4.2 Solution

Below are details of the features added to Jaltantra ,which enhances application monitoring which help in accelerated resolution of above issues.

- Utilizing Bash Scripting for Continuous Monitoring: Bash scripting is employed to check the status of endpoints continuously, ensuring that any issues or errors are detected in real-time. By automating the monitoring process, developers can receive timely alerts about potential issues, enabling them to respond promptly and minimize downtime.
- Sending Reports to Developers Every 24 Hours: In addition to real-time monitoring, reports summarizing the status of the application and any detected issues are sent to developers every 24 hours. These reports include details about errors found in the results folder of the deployment, providing developers with a comprehensive overview of the application's performance and any ongoing issues. By receiving regular reports, developers can stay informed about the application's health and address any persistent or recurring issues proactively.
- Utilizing cron Job for Scheduled Tasks: cron job, a time-based job scheduler in Unix-like operating systems, is utilized to schedule and automate the execution of monitoring scripts and report generation. By leveraging cron job functionality, monitoring tasks and report generation are executed at predefined intervals without manual intervention. This ensures that developers receive timely reports and alerts without the need for constant monitoring, allowing them to focus on addressing issues and improving application stability.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The initial work started with thoroughly understanding the currently deployed Jaltantra System. The code base is large and complex to understand, so initial time was devoted to understand each component properly. Each architectural component has been discussed and described in detail.

After carefully analyzing the code-base , it was found to have limitations like non-modularity in code-base, dependency on outdated libraries,dependency on IDE for packaging and compiling the code base, manual deployment process,lack of application level security and user management etc, also due to above limitations fixing of bugs or vulnerabilities which had been discovered was proving to be difficult.

Even a single change in the project requires a lot of efforts due to low code modularity, so refactoring of the code must also be done to bring modularity, so initially code was refactored and then migrated to spring framework , for rapid development process spring boot was used.

After migration to spring boot bugs were resolved and application was configurable by using external files, various features of spring like maven build and dependency management,autowiring etc were utilized to simplify deployment, building and development process.

Features like user management were added to make application secure and helping it to track the users using the services.Branch and Loop version are now packaged and run as a standalone application without any dependency on external servers.

6.2 Future Work

The current code base after migration is extensible , making feature enhancement process easier. currently deployment and monitoring automation is done by using bash features, however if need arise then spring eco-system provides various tools for these deployment and monitoring as well , these can also be integrated as well with minimal developer effort.

Continous integration and deployment can also be integrated given , deployment and build process have been simplified

Based on user engagement in future if required application can be migrated to micro services based architecture, containerization etc can be used to ensure dependency management and auto scaling for CPU intensive python based solver back-end.

For micro-services main micro-services identified as per current code-base

- Front end user interface which is JavaScript based can be 1 service interacting with middle-ware.
- Middle-ware (2nd micro-service) to process data which is java based can be second service , and middle ware interacting via rest-API's with back-end.
- Python back-end micro-service (3rd micro-service) which has CPU intensive workloads to handle, which can run on IITB-servers with solver license and return the solution to inputs provided by middle-ware.

Bibliography

- [1] Apache Software Foundation. Tomcat user guide. https://tomcat.apache.org/tomcat-3.2-doc/uguide/tomcat_ug.html, 2001.
- [2] Apache Software Foundation. Maven guides. <https://maven.apache.org/guides/index.html>, 2024.
- [3] Django. Django framework. <https://docs.djangoproject.com/en/5.0/>, 2024.
- [4] Nikhil Hooda and Om Damani. Jaltantra: A system for the design and optimization of rural piped water networks. *INFORMS Journal on Applied Analytics*, 49(6):447–458, 2019.
- [5] Jakarta EE. Jakarta ee. <https://jakarta.ee/specifications/platform/10/>, 2024.
- [6] JetBrains. IntelliJ support request 5446484. <https://intellij-support.jetbrains.com/hc/en-us/requests/5446484>, 2024.
- [7] Node JS. Node js. <https://nodejs.org/docs/latest/api/>, 2024.
- [8] Oracle Corporation. Mysql documentation. <https://dev.mysql.com/doc/>, 2024.
- [9] Pivotal Software, Inc. Spring boot reference documentation. <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>, 2024.